Введение в списки

Список - это непрерывная динамическая коллекция элементов. Каждому элементу списка присваивается порядковый номер - его индекс. Первый индекс равен нулю, второй - единице и так далее. Основные операции для работы со списками - это индексирование, срезы, добавление и удаление элементов, а также проверка на наличие элемента в последовательности.

СЦ **Sl Tsekhosh**

Дата обращения: 09.11.2025

Создание и основы работы со списками

1 Создание списка

Списки в Python создаются с помощью квадратных скобок [] или функции list(). Каждый элемент списка может быть любым типом данных, включая числа, строки, булевы значения и даже другие объекты.

2 Доступ к элементам

Элементы списка индексируются, начиная с 0. Вы можете обращаться к ним с помощью их индексов, например, my_list[0] для получения первого элемента.

3 Изменение и добавление элементов

Списки являются изменяемыми, что означает, что вы можете добавлять, удалять и изменять элементы по мере необходимости, используя встроенные методы.



Создание пустого списка

```
my_list = []       # пустой список
my_list = list()   # пустой список
```

Создание списка, состоящего из целых чисел:

```
numbers_list = [40, 20, 90, 11, 5]
```

Создание списка, состоящего из дробных чисел:

```
numbers_list = [3.14, 2.72, 1.41, 1.73, 17.9]
```

Создание списка, состоящего из строковых переменных:

```
car_models_list = ['Lada Granta','Lada Vesta','Niva Travel','УАЗ Патриот','Aurus Senat','Lada Largus']
```

Создание списка, состоящего из разных переменных:

```
my_list _list = [3.14, 10, 'Hello world!', False, 'Python is the best']
```

Индексация

```
          0              1              2              3              4
car_models = ['Lada Granta', 'Niva Travel', 'Lada Vesta', 'УАЗ Патриот', 'Aurus Senat']
print(car_models[0])
print(car_models[1])
print(car_models[4])

#Вывод
Lada Granta
Niva Travel
Aurus Senat
```



Индексация операция обращения к элементу по его порядковому номеру. Списки в Python индексируются, начиная с **0**. Вы можете обращаться к элементам списка по их индексам, чтобы получать, изменять или заменять их.

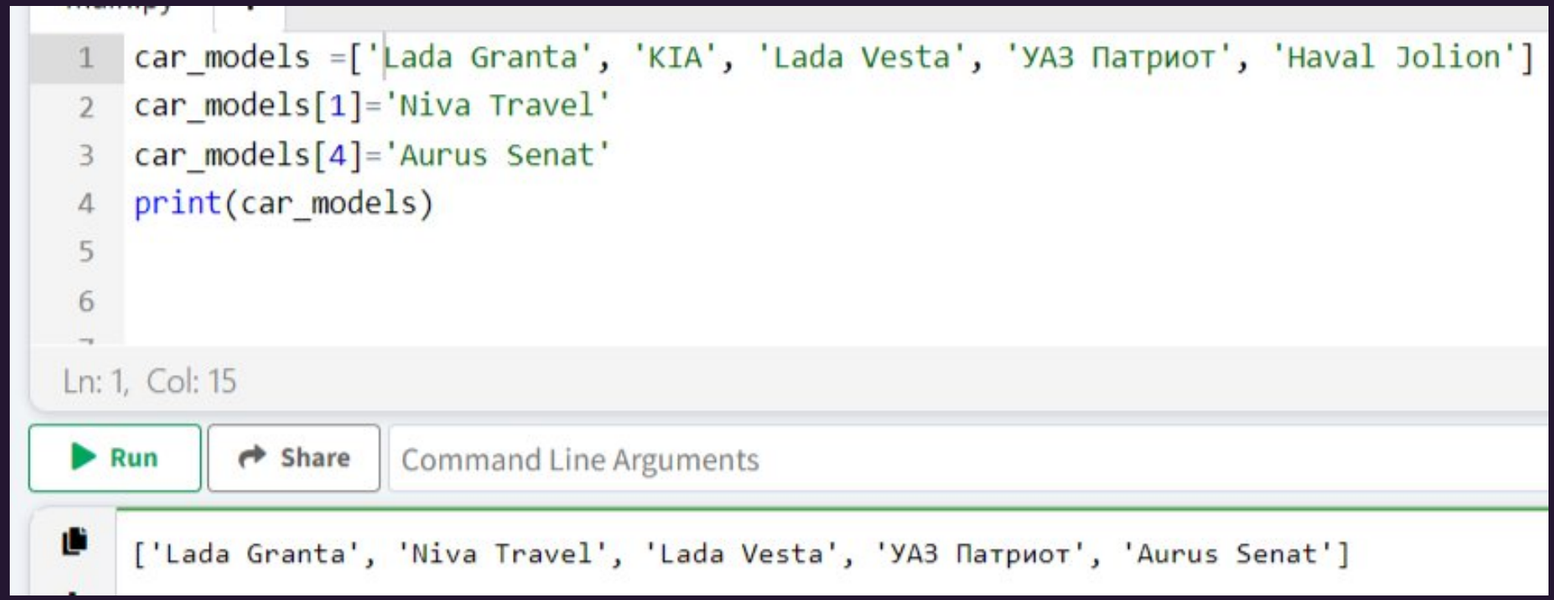
Индексация. Списки в Python являются **изменяемым** типом данных.

```
car_models  = ['Lada Granta', 'KIA', 'Lada Vesta', 'УАЗ Патриот', 'Haval Jolion']
                0             1      2             3              4

car_models[1] = 'Niva Travel'

car_models[4] = 'Aurus Senat'

print(car_models)
```



#Вывод

['Lada Granta', '**Niva Travel**', 'Lada Vesta', 'УАЗ Патриот', '**Aurus Senat**']

Индексация операция обращения к элементу по его порядковому номеру. Списки в Python индексируются, начиная с **0**. Вы можете обращаться к элементам списка по их индексам, чтобы получать, изменять или заменять их.

Индексация. Индексирование работает и в обратную сторону

```
                -4              -3              -2          -1
car_models  = [' Lada Granta',  'Lada Vesta', 'УАЗ Патриот', 'Haval Jolion']

print(car_models[-1])

print(car_models[-2])

print(car_models[-3])

print(car_models[-4])
```

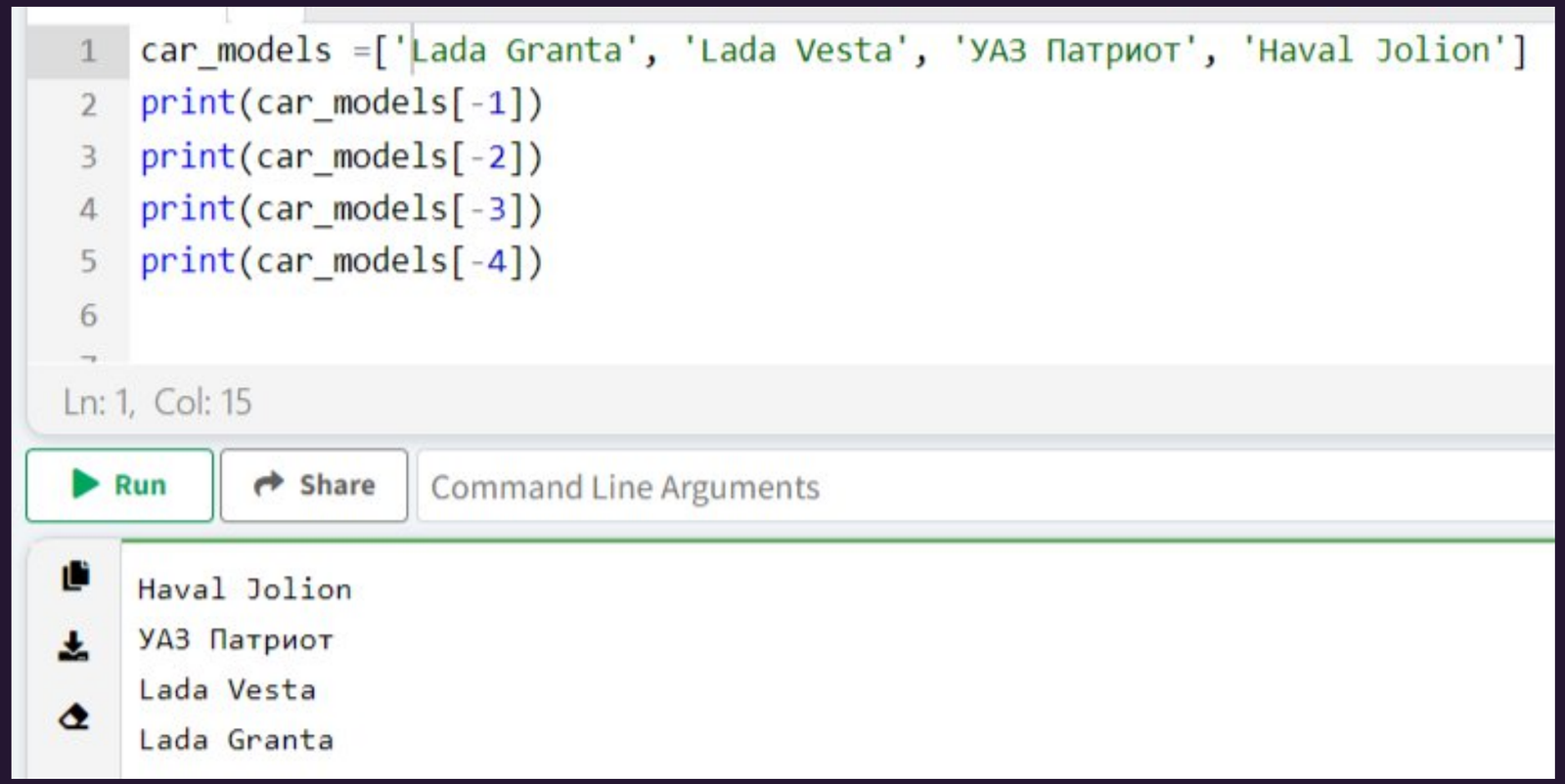
#Вывод

Haval Jolion

УАЗ Патриот

Lada Vesta

Lada Granta



Индекс с номером **-1** дает нам доступ к **последнему** элементу, -2 к предпоследнему и т.д.

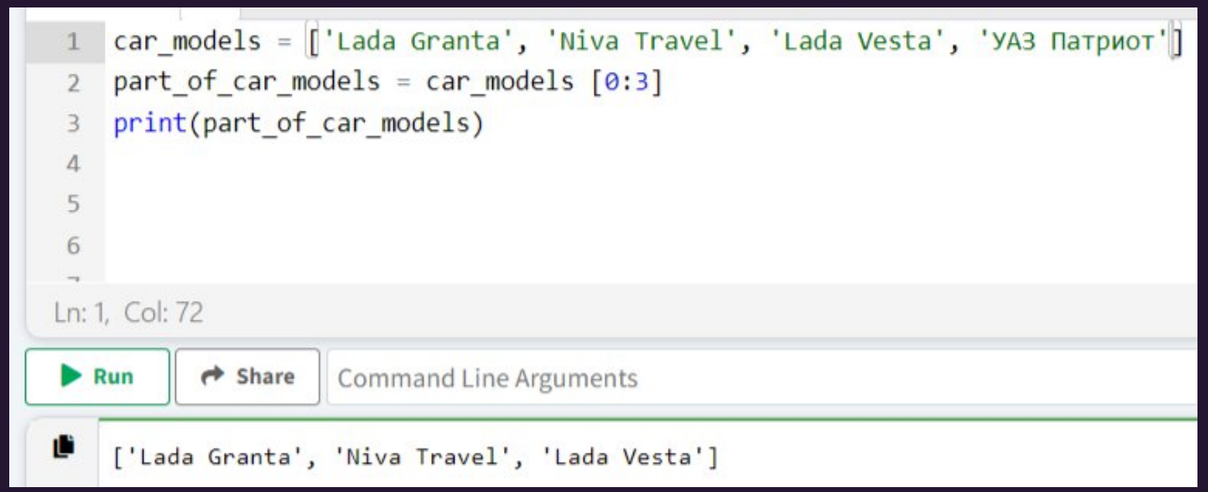
Срез списка

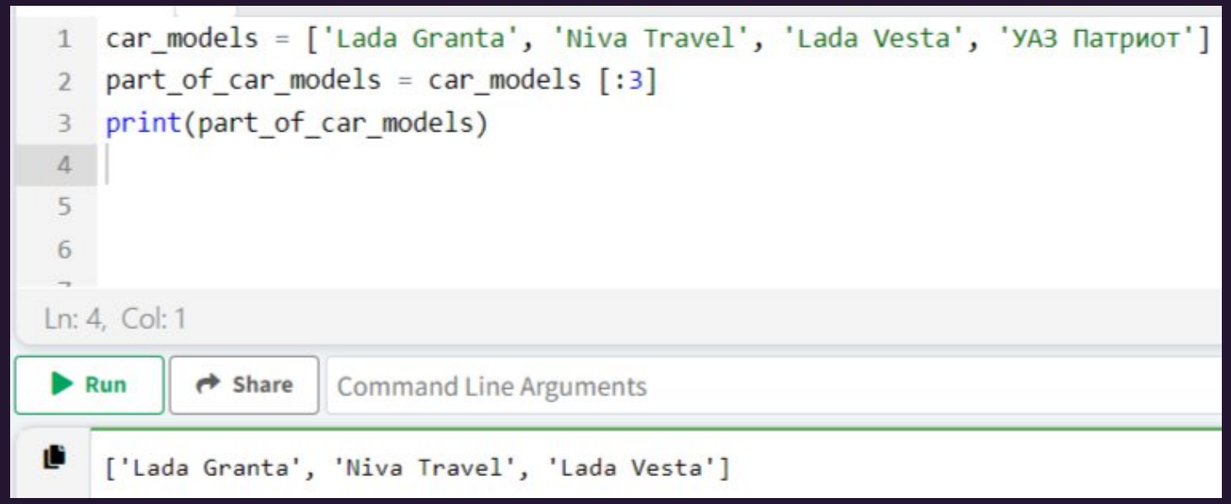
```
                0          1          2          3
car_models = ['Lada Granta', 'Niva Travel', 'Lada Vesta', 'УАЗ Патриот']
part_of_car_models = car_models [0:3]
print(part_of_car_models)

#Вывод
['Lada Granta', 'Niva Travel', 'Lada Vesta']
```

```
                0          1          2          3
car_models = ['Lada Granta', 'Niva Travel', 'Lada Vesta', 'УАЗ Патриот']
part_of_car_models = car_models [:3]
print(part_of_car_models)

#Вывод
['Lada Granta', 'Niva Travel', 'Lada Vesta']
```



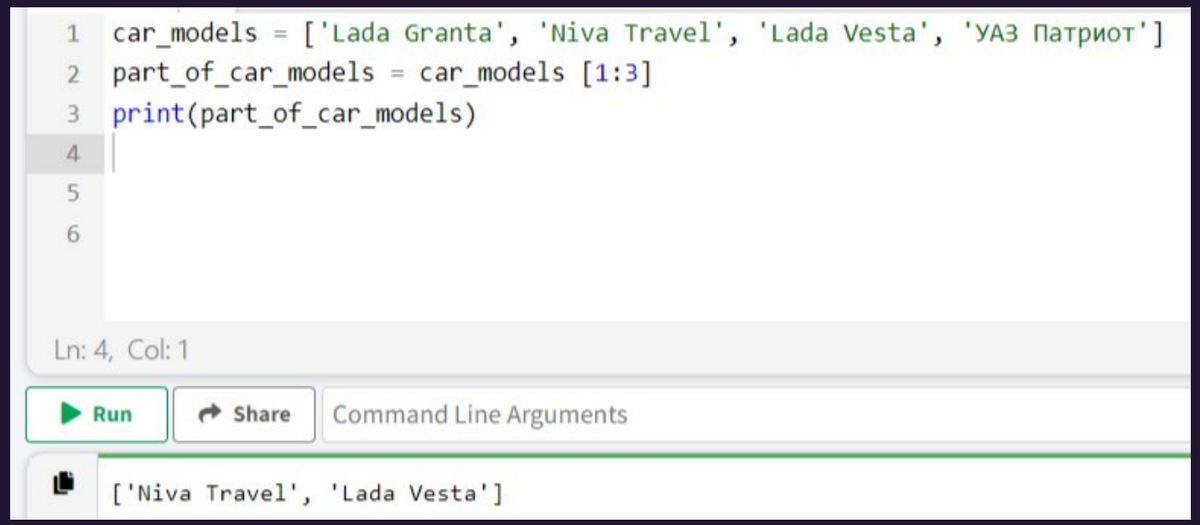


Срезом называется некоторая подпоследовательность. "Выбираем" часть от исходной последовательности элемента, не меняя её при этом. Срезы могут быть полезны при работе с большими массивами данных, когда необходимо быстро выбрать часть данных для обработки или ана

Срез списка

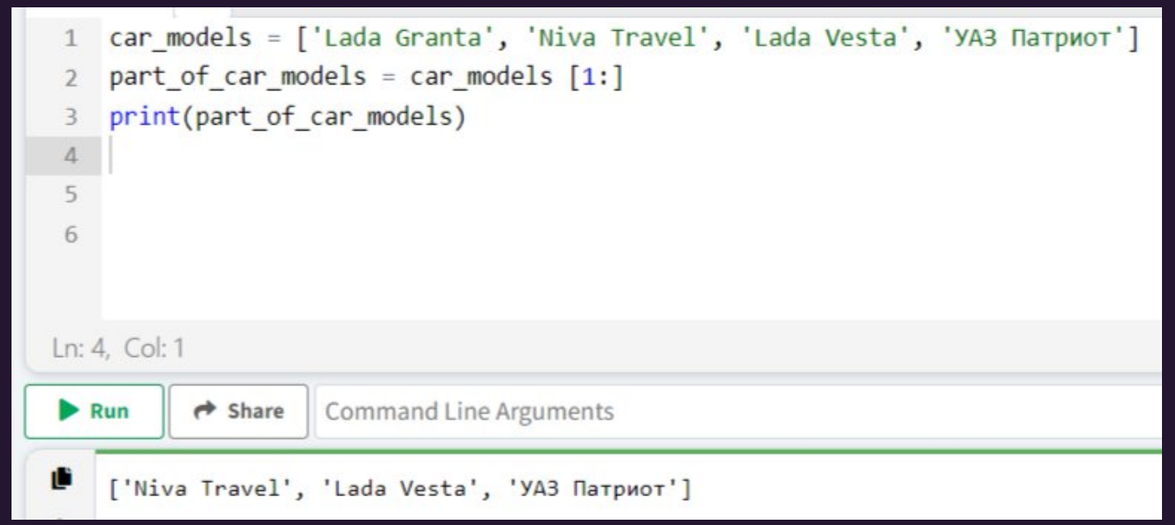
```
                    0           1           2           3
car_models = ['Lada Granta', 'Niva Travel', 'Lada Vesta', 'УАЗ Патриот']
part_of_car_models = car_models [1:3]
print(part_of_car_models)

#Вывод
['Niva Travel', 'Lada Vesta']
```




```
                        0           1           2           3
car_models = ['Lada Granta', 'Niva Travel', 'Lada Vesta', 'УАЗ Патриот']
part_of_car_models = car_models [1:]
print(part_of_car_models)

#Вывод
['Niva Travel', 'Lada Vesta', 'УАЗ Патриот']
```



Срезом называется некоторая подпоследовательность. "Выбираем" часть от исходной последовательности элемента, не меняя её при этом.

Срез списка

car_models[2:4:1]



✓ Синтаксис среза. Итерируемая_переменная [начальный_индекс: конечный_индекс - 1: длина_шага].

```
                    0      1      2          3             4
car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']

print(car_models[2:4:1])

print(car_models[1:3:1])


#Вывод

>>> ['KIA', 'Москвич']

>>> ['УАЗ', 'KIA']
```

```
1  car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']
2  print(car_models[2:4:1])
3  print(car_models[1:3:1])
4
5
6

Ln: 4, Col: 1

Run   Share   Command Line Arguments

['KIA', 'Москвич']
['УАЗ', 'KIA']
```

Срезом называется некоторая подпоследовательность. "Выбираем" часть от исходной последовательности элемента, не меняя её при этом.

Срез списка

car_models[2:4:1]

- ✓ Синтаксис среза. Итерируемая_переменная [начальный_индекс: конечный_индекс - 1: длина_шага].
- ✓ Длина шага тоже может быть **отрицательной**!

```
                -5      -4      -3          -2          -1
car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']

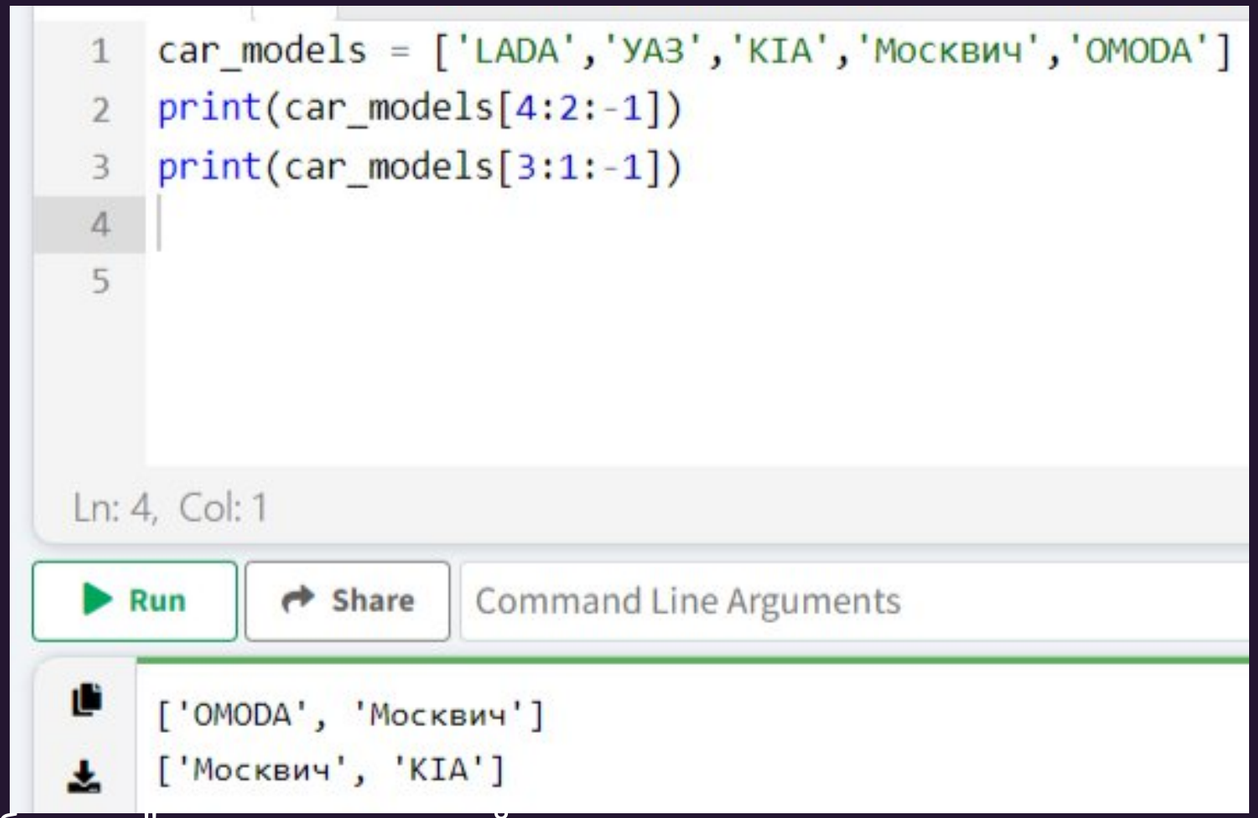
print(car_models[4:2:-1])

print(car_models[3:1:-1])


#Вывод

>>> ['OMODA', 'Москвич']

>>> ['Москвич', 'KIA']
```



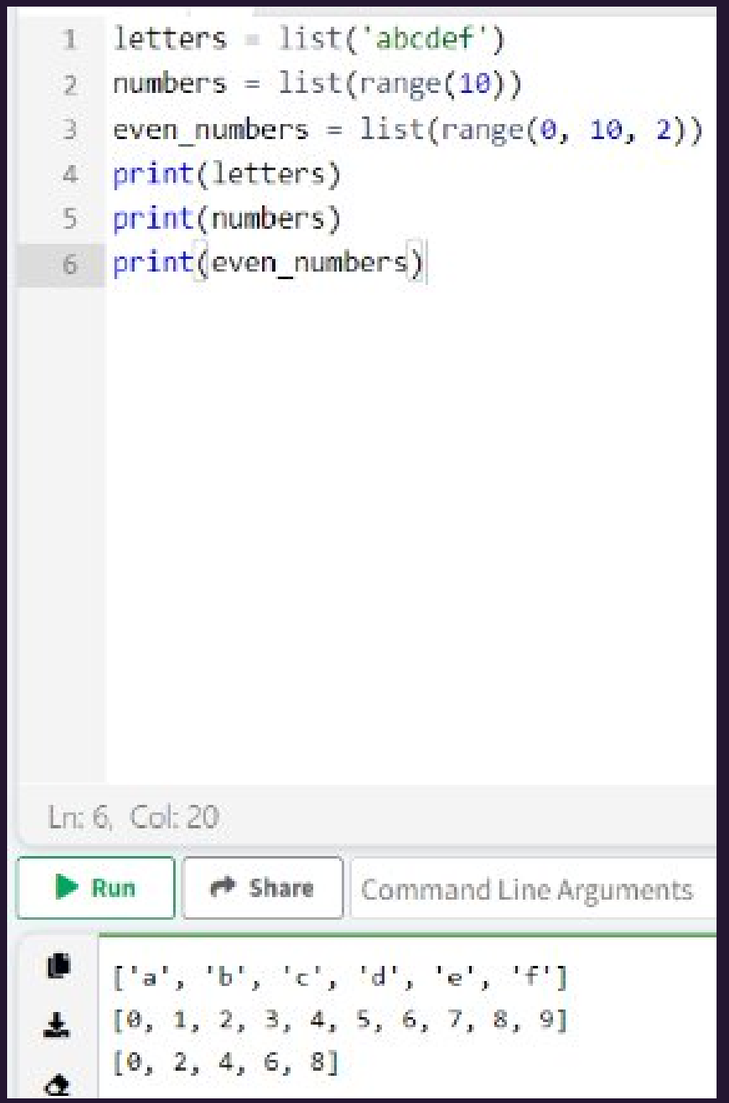
Срезом называется некоторая подпоследовательность. "Выбираем" часть от исходной последовательности элемента, не меняя её при этом.

Создание списка с помощью функции list()

```
letters = list('abcdef')
numbers = list(range(10))
even_numbers = list(range(0, 10, 2))
print(letters)
print(numbers)
print(even_numbers)
```

#Вывод

```
>>> ['a', 'b', 'c', 'd', 'e', 'f']
>>> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [0, 2, 4, 6, 8]
```



Длина списка и проверка принадлежности

Функция len()

Функция len() возвращает количество элементов в списке. Она очень полезна для определения размера списка и итерации по его элементам.

Оператор in

Оператор in позволяет проверить, содержится ли определенный элемент в списке. Он возвращает True, если элемент найден, и False в противном случае.

Длина списка

```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
print(len(fruits))
```

#Вывод

5

Функция len() возвращает **длину** любой итерируемой переменной, переменной, по которой можно запустить цикл.

Оператор in **Функция len()** Цикл **for**

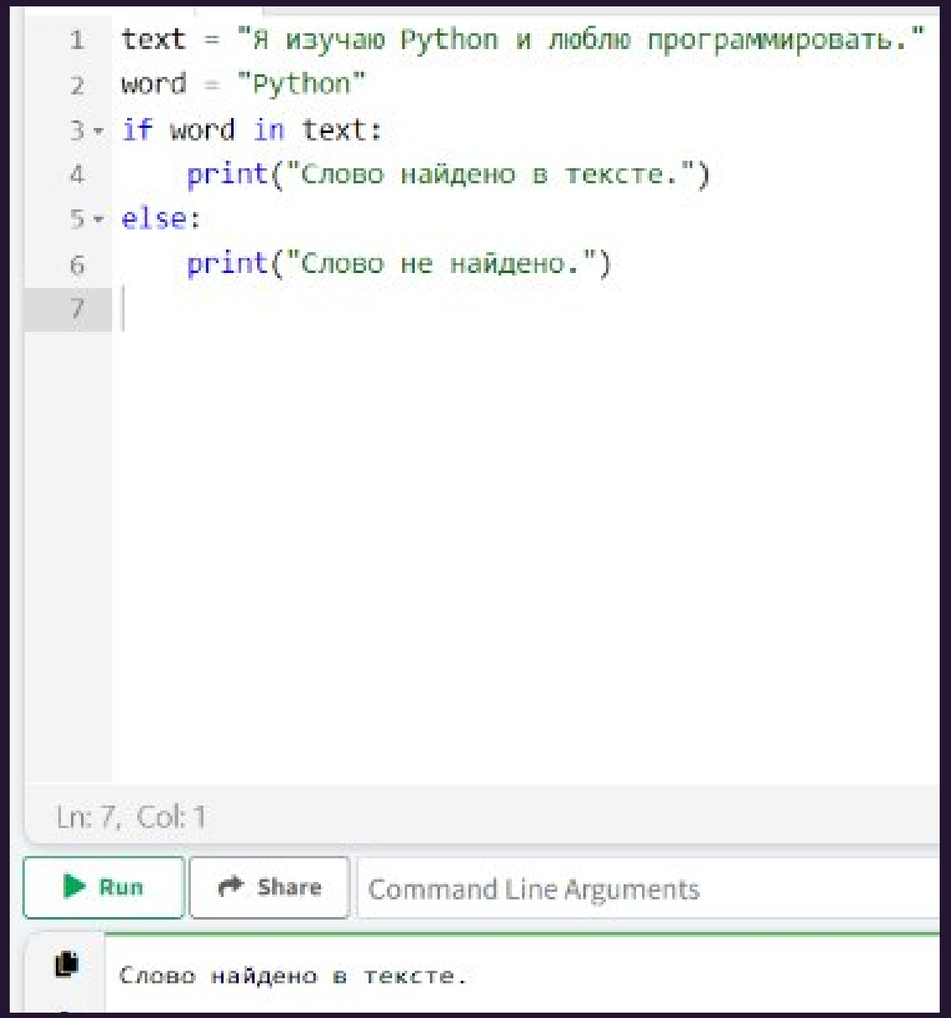
```
names = ['Павел', 'Петр', 'Анна']
for name in names:
    print('Имя' ,name, 'имеет длину', len(name))


#Вывод
Имя Павел имеет длину 5
Имя Петр имеет длину 4
Имя Анна имеет длину 4
```

Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в **списке**, строке или кортеже).

Оператор in

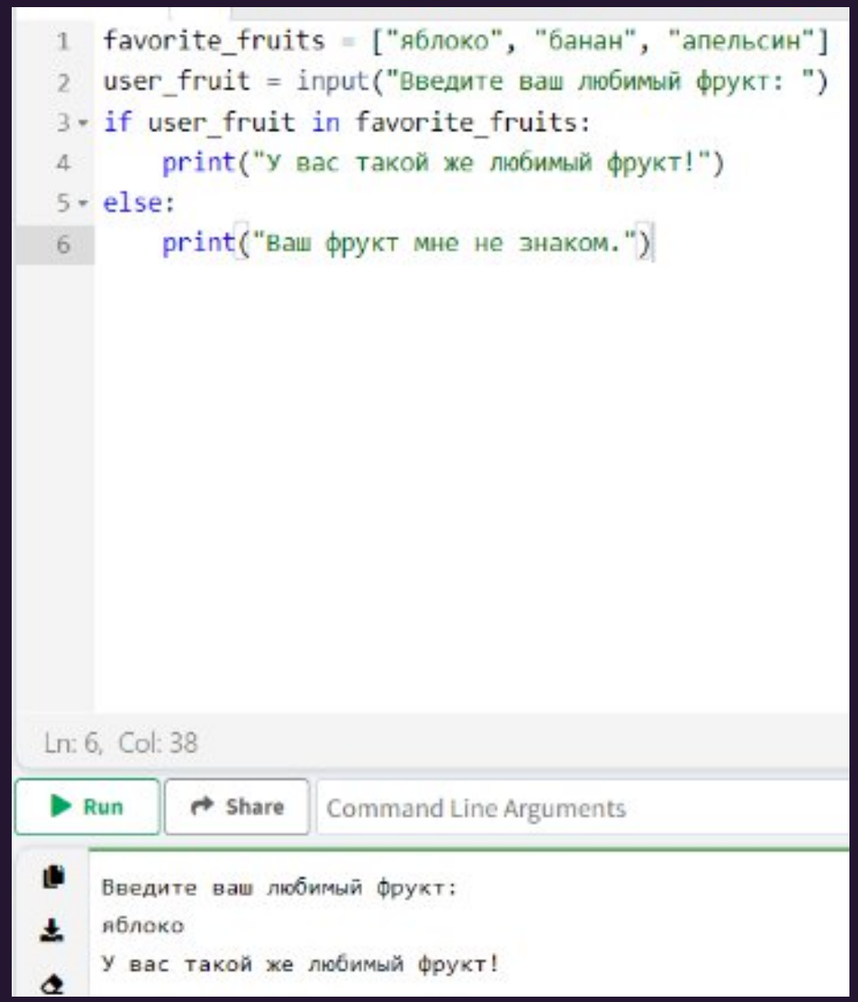
```
text = "Я изучаю Python и люблю программировать."
word = "Python"
if word in text:
    print("Слово найдено в тексте.")
else:
    print("Слово не найдено.")
```



Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в списке, строке или кортеже). Если элемент присутствует, оператор возвращает **True**, в противном случае — **False**.

Оператор in

```
favorite_fruits = ["яблоко", "банан", "апельсин"]
user_fruit = input("Введите ваш любимый фрукт: ")
if user_fruit in favorite_fruits:
    print("У вас такой же любимый фрукт!")
else:
    print("Ваш фрукт мне не знаком.")
```



Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в **списке**, строке или кортеже).

```
fruits = ['яблоко','апельсин','груша','ананас']

bakery_products = ['батон','булочка','хлеб','пироженое']

products = ['молоко','хлеб','яблоко','соль']


print('фрукты:')

for i in products:

    if i in fruits:

        print(i)


print('хлебобулочные изделия: ')

for i in products:

    if i in bakery_products:

        print(i)
```

▶ Содержимое файла `file.txt`

```
1  fruits = ['яблоко','апельсин','груша','ананас']
2
3  bakery_products = ['батон','булочка','хлеб','пироженое']
4
5  products = ['молоко','хлеб','яблоко','соль']
6
7  print('фрукты:')
8  for i in products:
9      if i in fruits:
10         print(i)
11
12 print('хлебобулочные изделия:')
13 for i in products:
14     if i in bakery_products:
15         print(i)
```

Выполнить!

```
фрукты:
яблоко
хлебобулочные изделия:
хлеб
```

Цикл **for** - удобный способ последовательно перебирать и выводить элементы списка. Он позволяет легко итерироваться по списку и обрабатывать каждый элемент.

Индексация и добавление элементов

1

Индексация

Списки в Python индексируются, начиная с 0. Вы можете обращаться к элементам списка по их индексам, чтобы получать, изменять или заменять их.

2

Добавление элементов

Для добавления новых элементов в список используются методы append() и insert(). Append() добавляет элемент в конец списка, а insert() позволяет вставить элемент в указанную позицию.

3

Применение

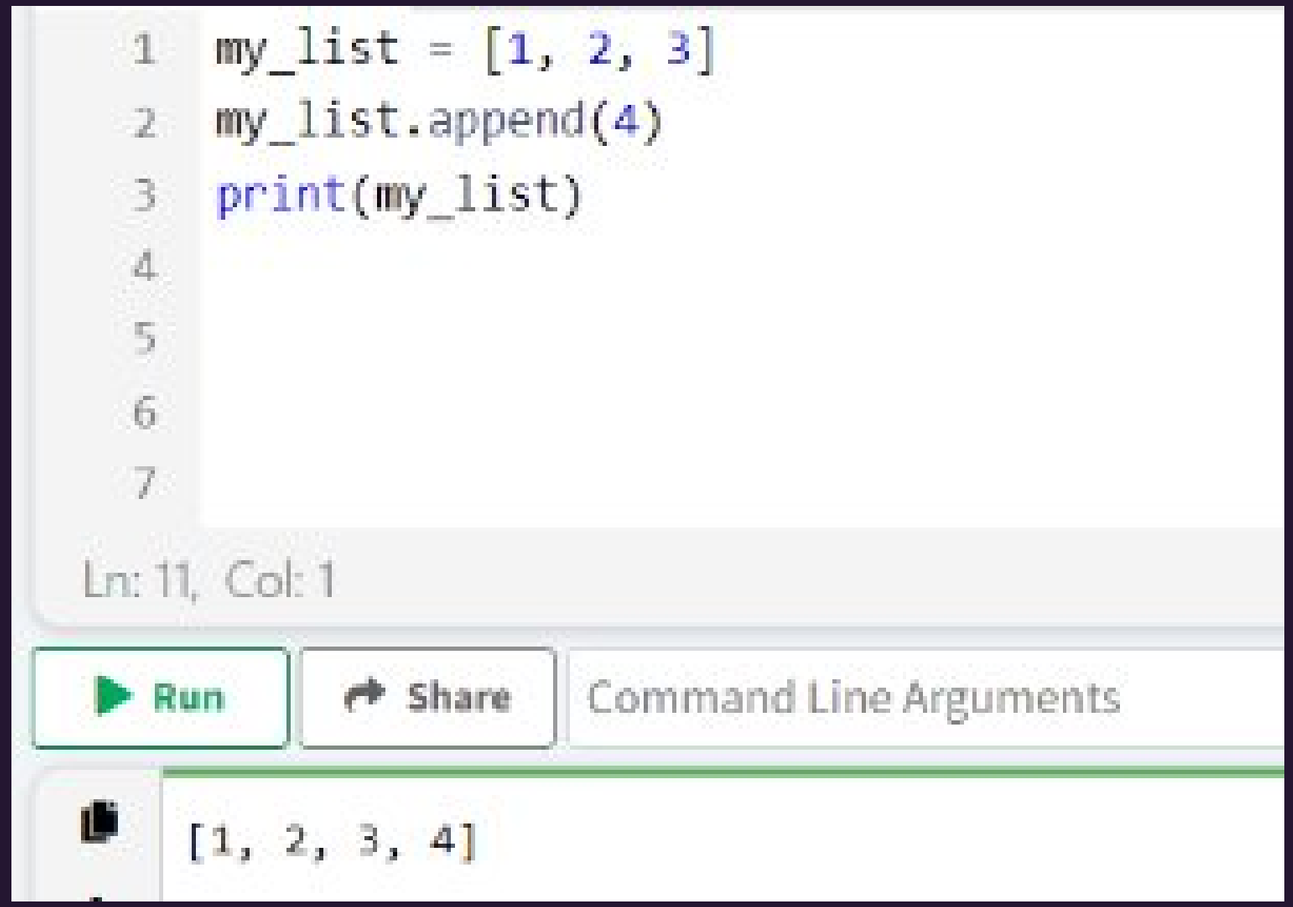
Индексация и добавление элементов - ключевые операции для манипулирования списками. Они позволяют вам создавать и изменять структуры данных, необходимые для решения ваших задач.



Метод append()

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)


#Вывод
[1, 2, 3, 4]
```

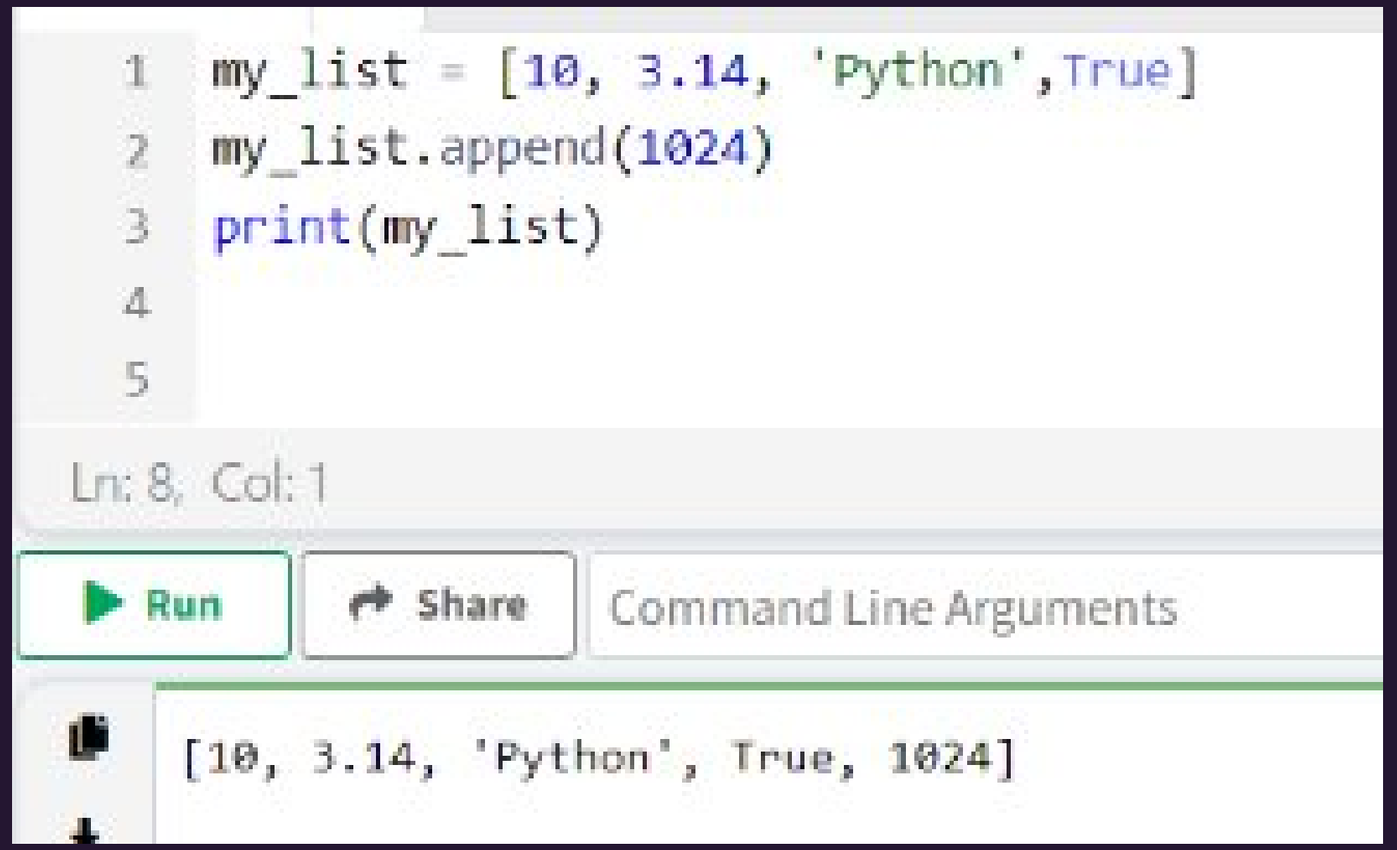


Метод **append()** - добавляет элемент в конец списка.

Метод **append()**

```
my_list = [10, 3.14, 'Python',True]
my_list.append(1024)
print(my_list)


#Вывод
[10, 3.14, 'Python', True, 1024]
```

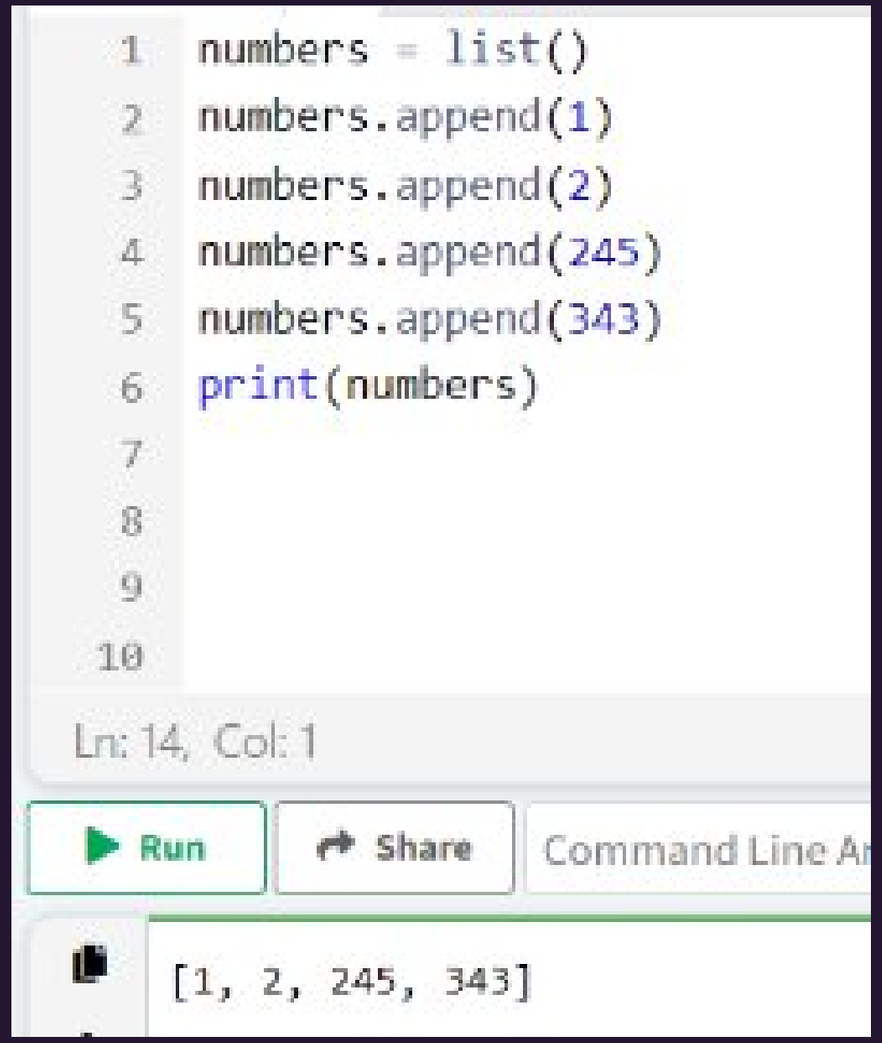


Метод **append()** - добавляет элемент в конец списка.

Метод **append()**

```
numbers = list()
numbers.append(1)
numbers.append(2)
numbers.append(245)
numbers.append(343)
print(numbers)


#Вывод
[1, 2, 245, 343]
```



Метод **append()** - добавляет элемент в конец списка.

Метод insert()

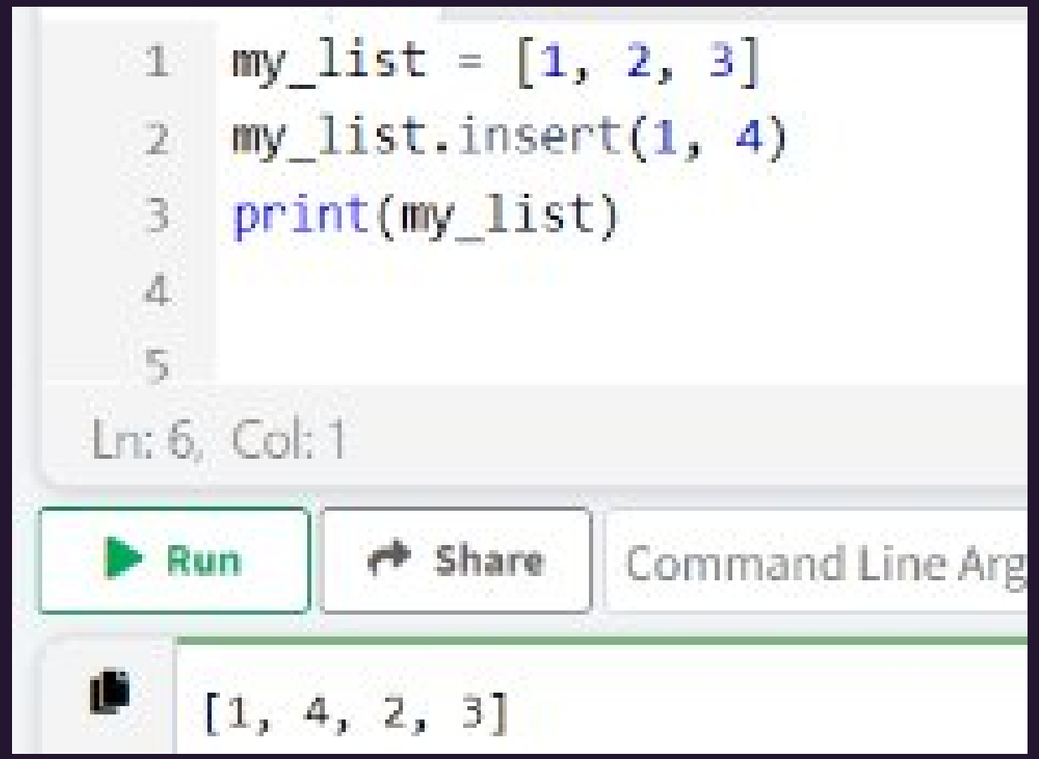
```
          0  1  2
my_list = [1, 2, 3]

my_list.insert(1, 4)

print(my_list)


# Вывод

[1, 4, 2, 3]
```



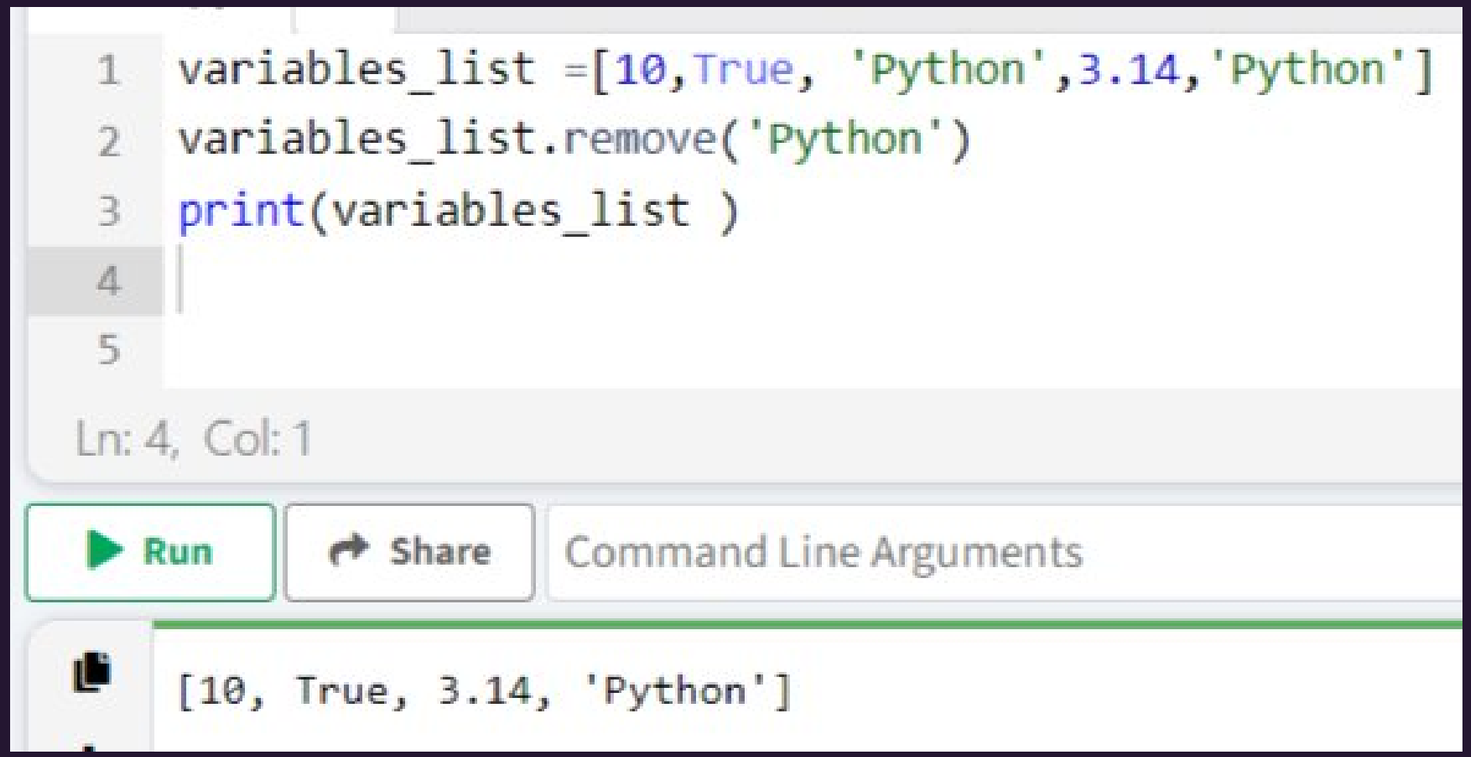
Метод **insert(**<index>, <value>**)** - добавляет элемент в список на заданную позицию.

insert() принимает в качестве первого аргумента индекс, по которому нужно вставить элемент, а вторым — сам элемент. Когда значение вставляется в список, длина списка увеличивается. Значение, которое ранее находилось в заданной индексной позиции, и все элементы после него сдвигаются на одну позицию к концу списка.

Метод remove()

```
variables_list =[10,True, 'Python',3.14,'Python']
variables_list.remove('Python')
print(variables_list )

#Вывод
[10, True, 3.14, 'Python']]
```



Метод remove() - удаляет первый элемент (пройдется по списку и удалит первое значение введенного элемента) по указанному значению.

Метод remove()

```
variables_list = [10, 'Python', 10, 3.14, 'Python']
variables_list.remove(3.14)
print(variables_list)
variables_list.remove(10)
print(variables_list )
variables_list.remove('Python')
print(variables_list )

# Вывод
>>> [10, 'Python', 10, 'Python']
>>> ['Python', 10, 'Python']
>>> [10, 'Python']
```



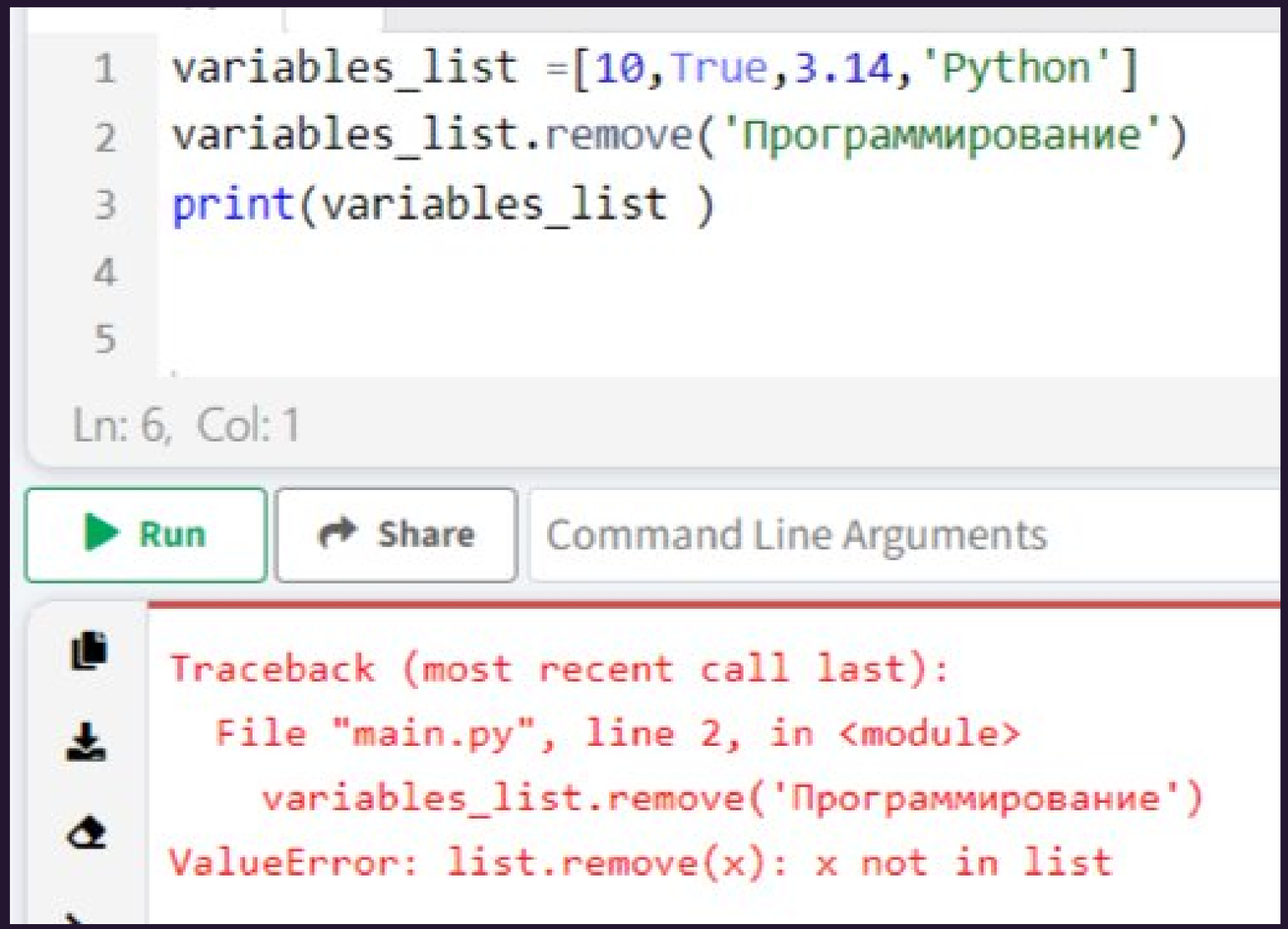
Метод remove() - удаляет первый элемент (пройдется по списку и удалит первое значение введенного элемента) по указанному значению.

Метод remove()

```
variables_list =[10,True,3.14,'Python']
variables_list.remove('Программирование')
print(variables_list )
```

#Вывод

```
Traceback (most recent call last):
File "main.py", line 2, in <module>
variables_list.remove('Программирование')
ValueError: list.remove(x): x not in list
```



Метод remove() - удаляет первый элемент (пройдется по списку и удалит первое значение введенного элемента) по указанному значению. Если элемент **не найден**, возникает исключение **ValueError**.

Метод pop()

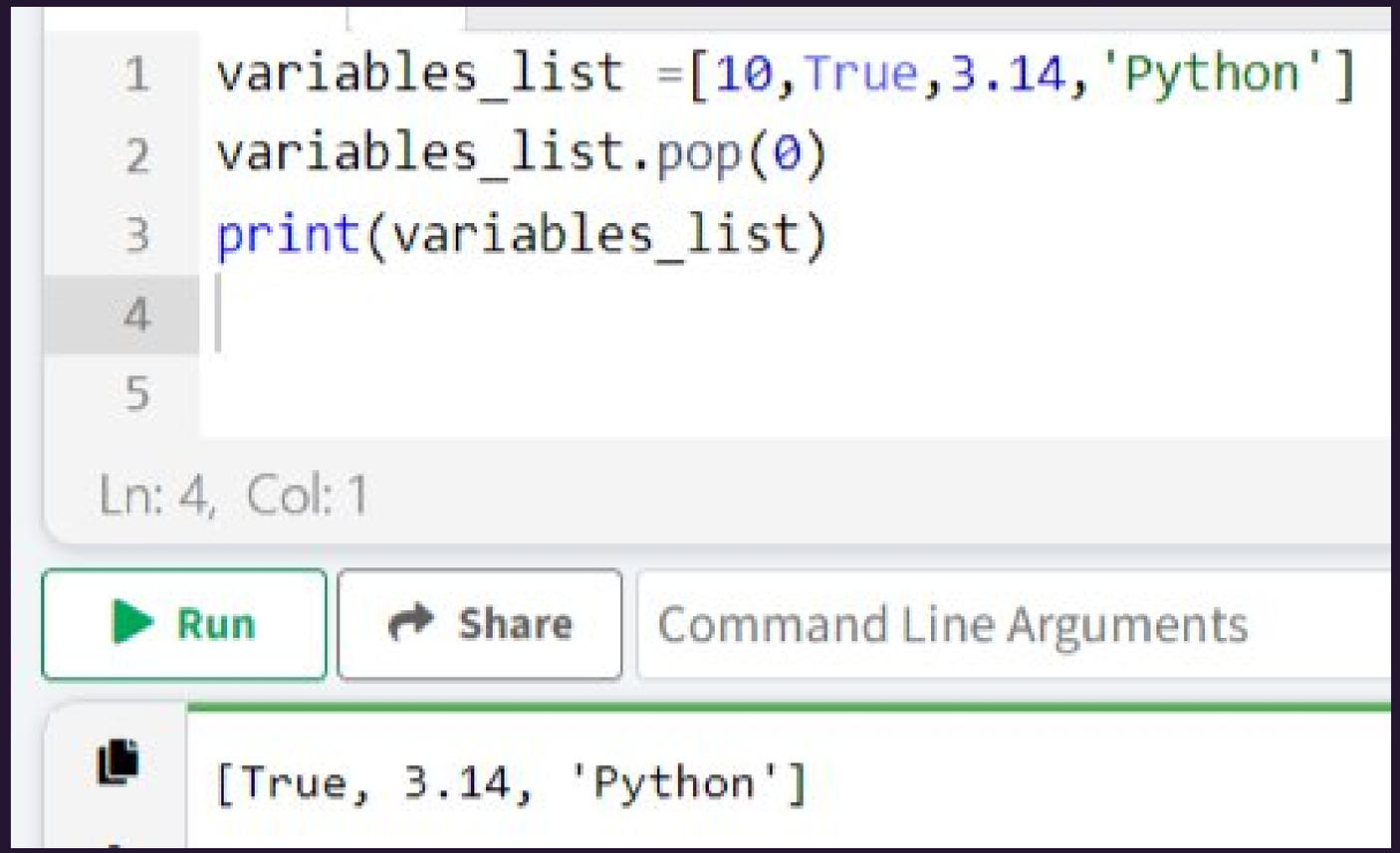
```
                0    1     2        3
variables_list = [10,True,3.14,'Python']

variables_list.pop(0)

print(variables_list)


#Вывод

[True, 3.14, 'Python']]
```



Метод pop() удаляет элемент из списка по его индексу. Удаляем первый элемент, если указан индекс.

Метод pop()

car_models = ['Lada Granta','Lada Vesta','**Niva Travel**']

car_models.**pop()**

print(car_models)

#Вывод

['Lada Granta', 'Lada Vesta']



Метод **pop()**, чтобы удалить **последний элемент**, вызовем метод **pop() без аргументов**.

Метод pop()

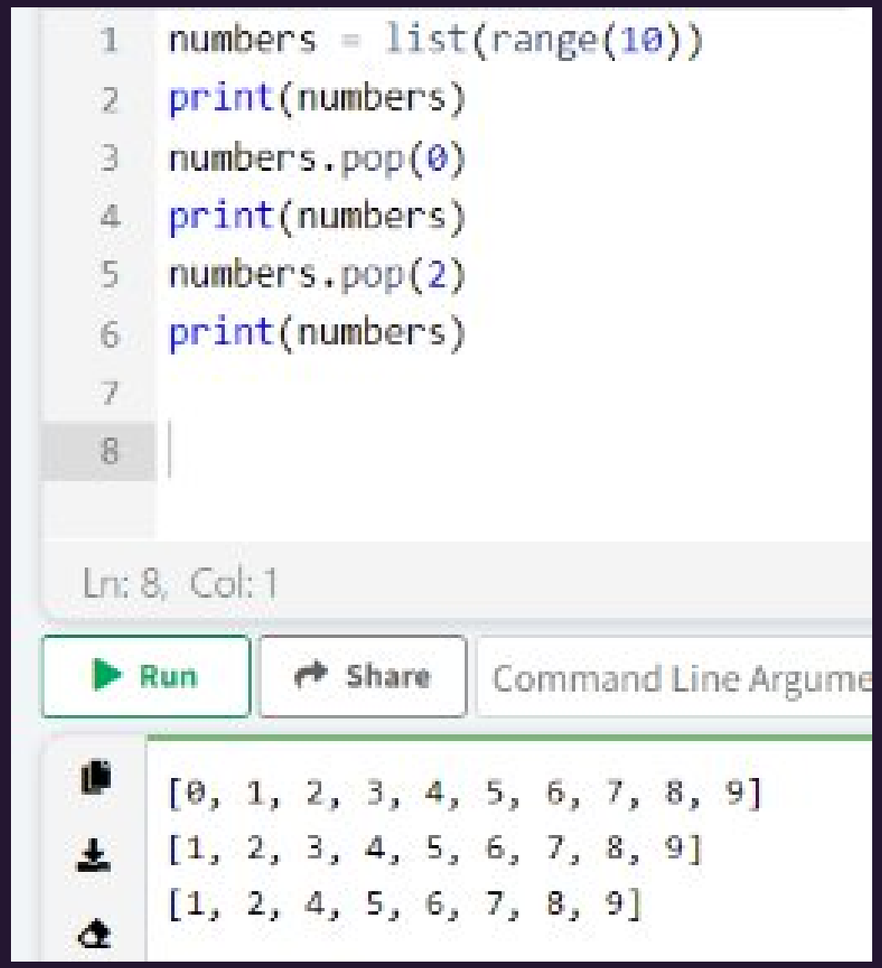
```
numbers = list(range(10))
print(numbers)
numbers.pop(0)
print(numbers)
numbers.pop(2)
print(numbers)
```

#Вывод

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[1, 2, 3, 4, 5, 6, 7, 8, 9]

[1, 2, 4, 5, 6, 7, 8, 9]



Метод pop() удаляет элемент из списка по его индексу. Удаляем первый элемент, если указан индекс.

Метод pop()

```
car_models = ['Lada Granta','Lada Vesta','Niva Travel']

new_car_models = car_models

car_models.pop()

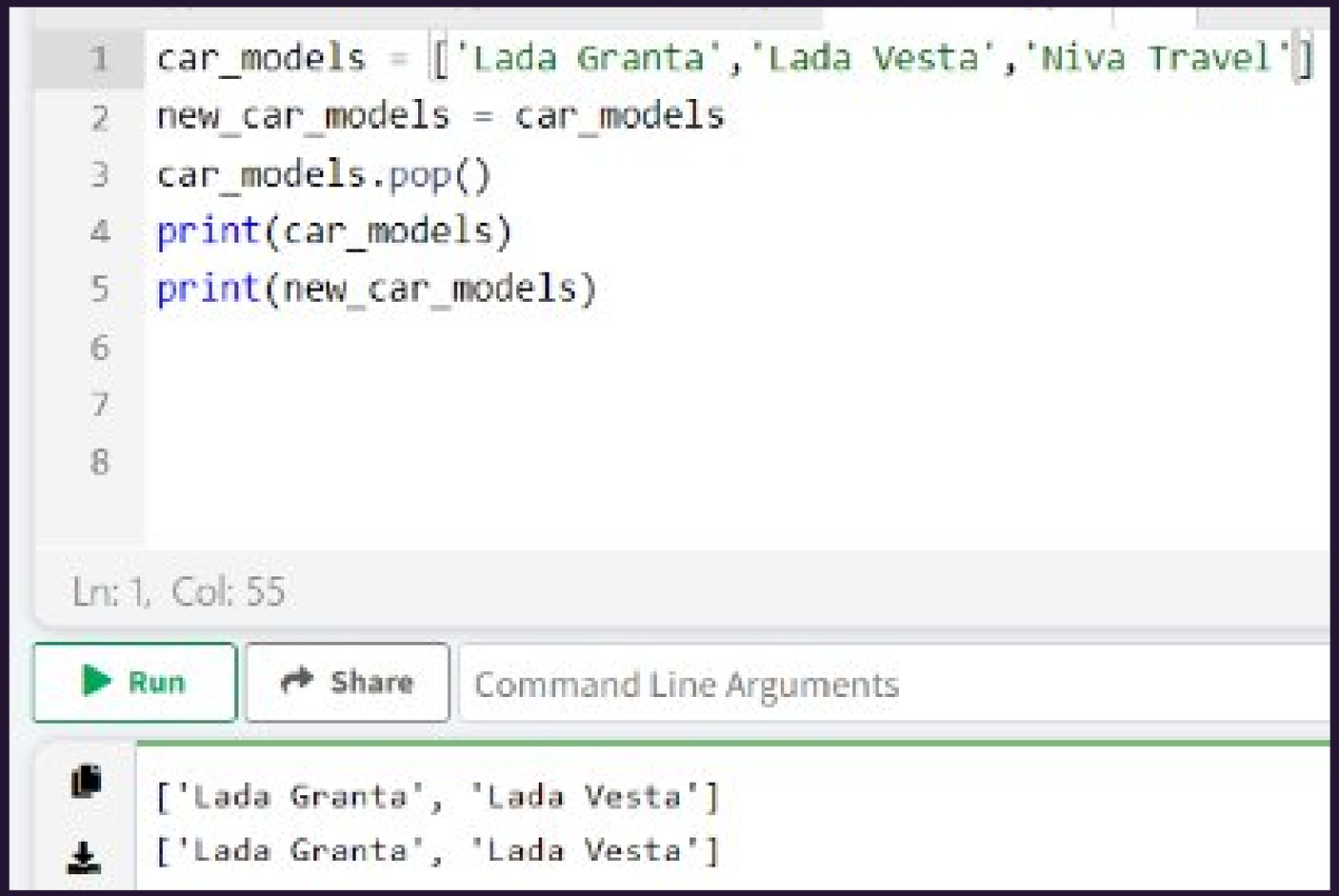
print(car_models)

print(new_car_models)


#Вывод

['Lada Granta', 'Lada Vesta']

['Lada Granta', 'Lada Vesta']
```



Метод **pop()**, чтобы удалить **последний элемент**, вызовем метод **pop() без аргументов**.

Метод clear()

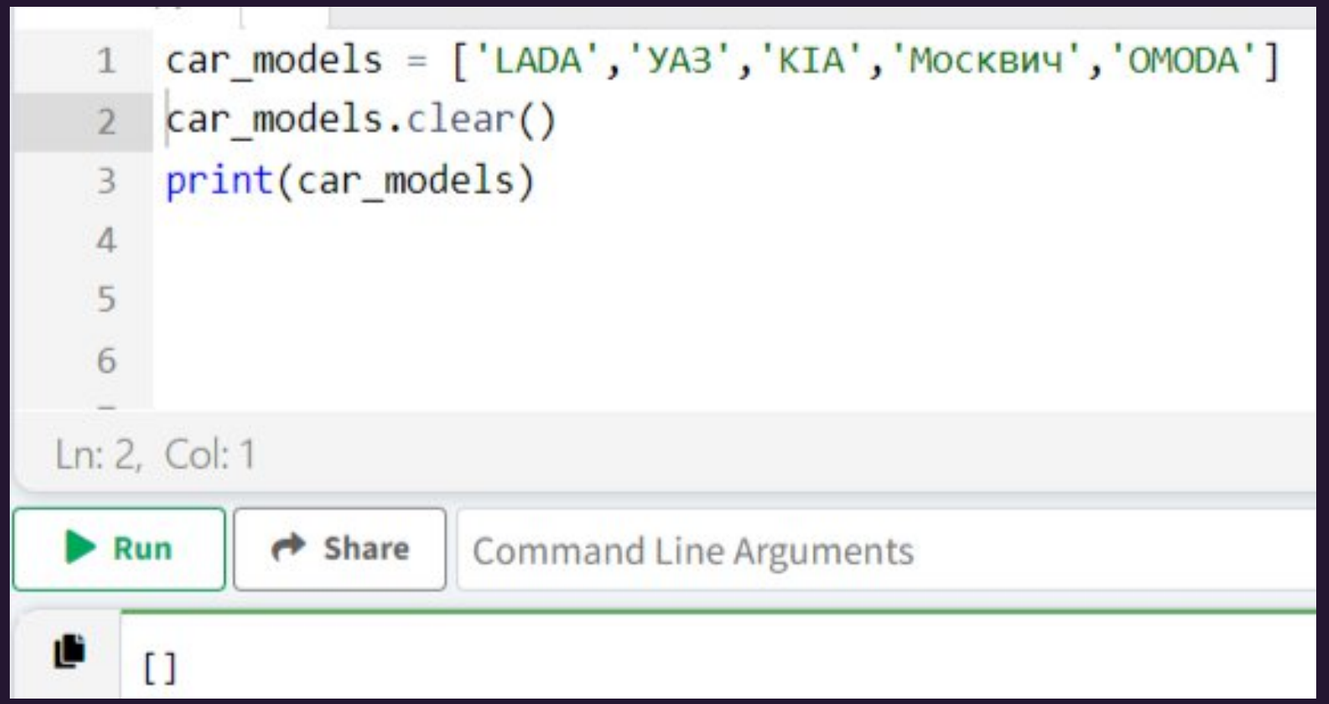
```
                    0       1       2           3           4
car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']

car_models.clear()

print(car_models)

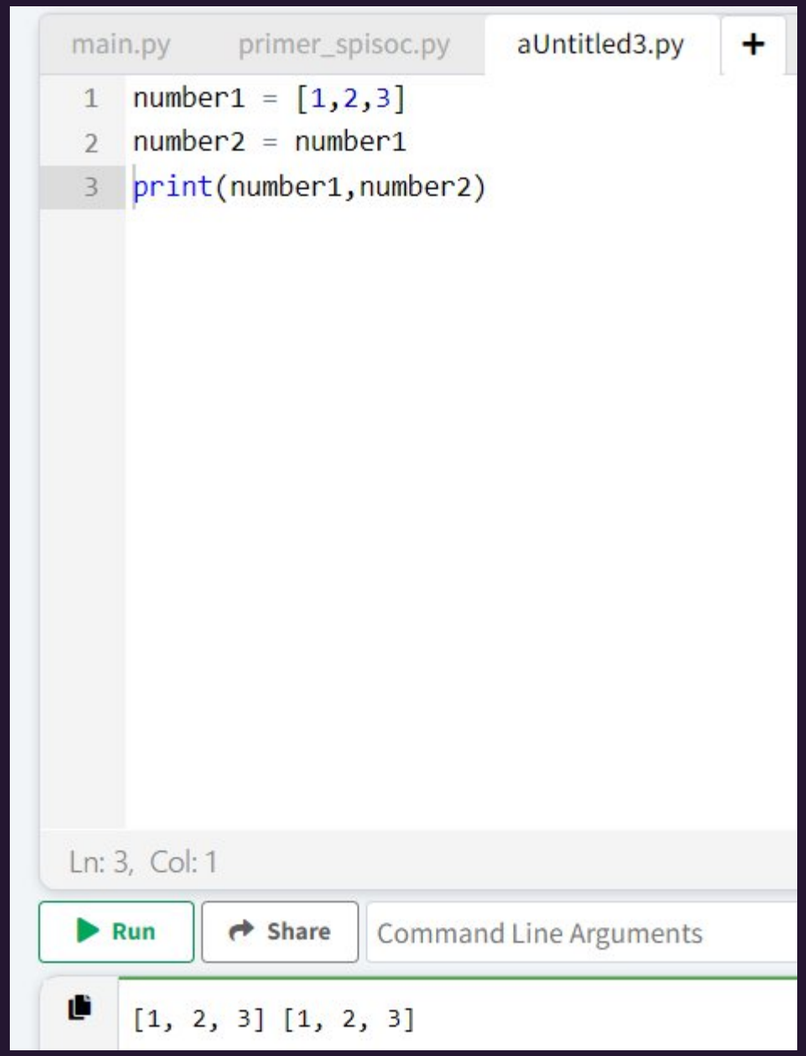

#Вывод

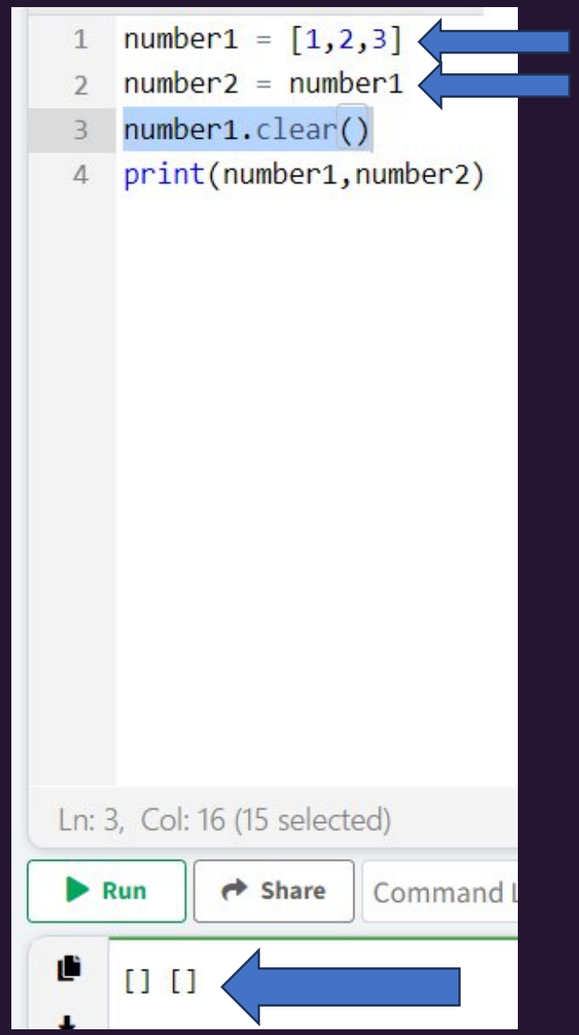
[]
```



Метод **clear()**- удаляет все элементы из списка, оставляя его пустым.

Метод clear()



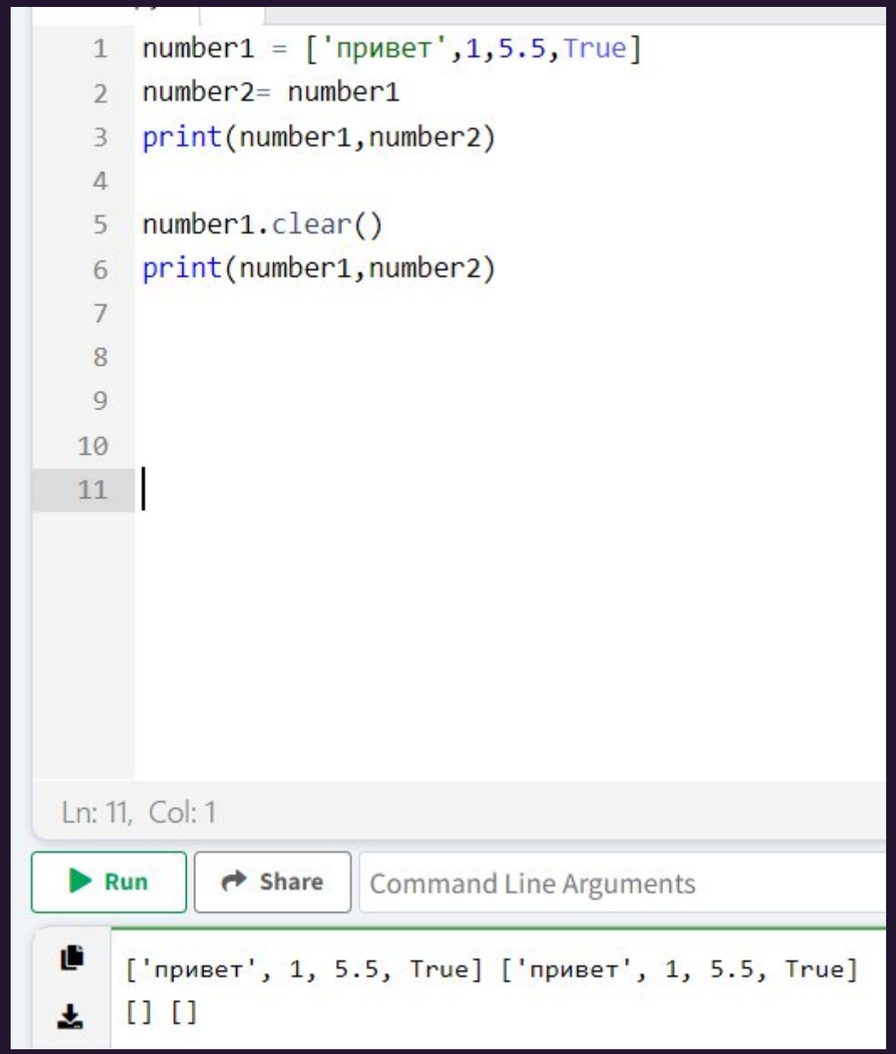


Метод **clear()**очищает сам список и везде где на него ссылаются он тоже очищается. Это удобно, когда вам нужно очистить список для дальнейшего использования.

Метод clear()

```
number1 = ['привет',1,5.5,True]
number2= number1
print(number1,number2)
number1.clear()
print(number1,number2)


#Вывод
['привет', 1, 5.5, True] ['привет', 1, 5.5, True]
[] []
```

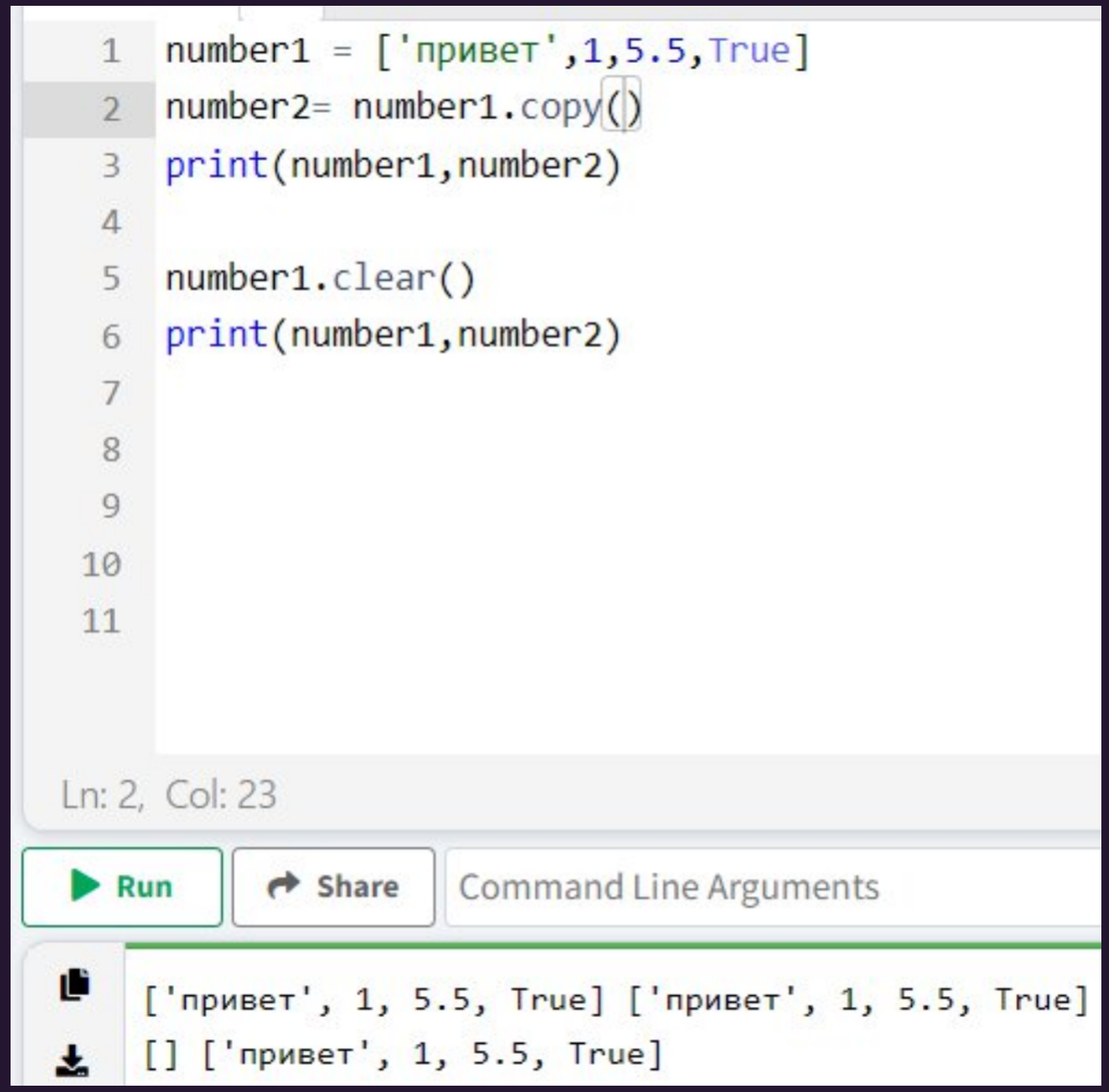


Метод clear()очищает сам список и везде где на него ссылаются он тоже очищается.

Метод clear()

```
number1 = ['привет',1,5.5,True]
number2= number1.copy()
print(number1,number2)
number1.clear()
print(number1,number2)

#Вывод
['привет', 1, 5.5, True] ['привет', 1, 5.5, True]
[] ['привет', 1, 5.5, True]
```

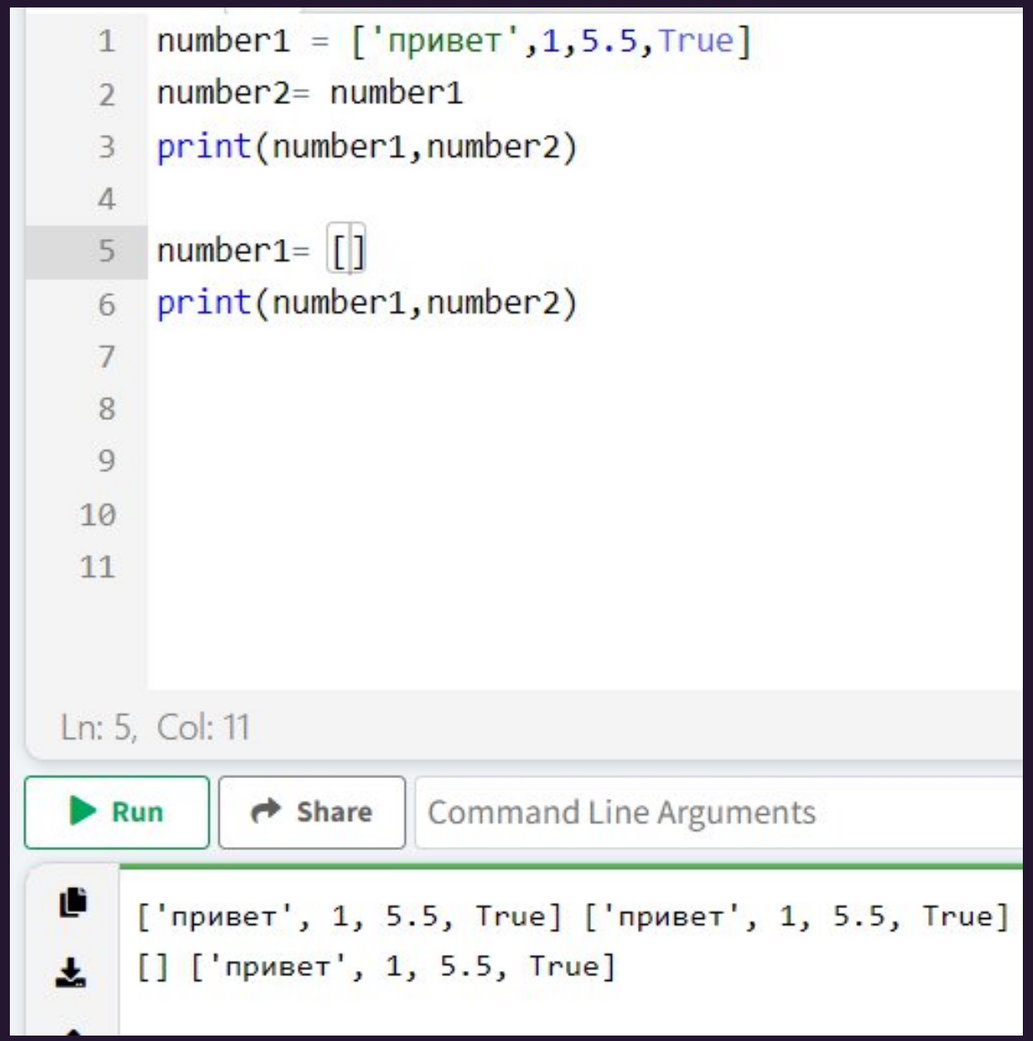


Метод **clear()** очищает сам список и везде где на него ссылаются он тоже очищается. Чтобы избежать данную ситуацию вызываем метод **copy()** при инициализации второго списка ([ ]).

Метод clear()

```
number1 = ['привет',1,5.5,True]
number2 = number1
print(number1,number2)
number1 = [ ]
print(number1,number2)

#Вывод
['привет', 1, 5.5, True] ['привет', 1, 5.5, True]
[] ['привет', 1, 5.5, True]
```



Метод **clear()** очищает сам список и везде где на него ссылаются он тоже очищается. Чтобы избежать данную ситуацию присваиваем пустой список [ ].

Метод clear()

[ ] если сравнивать с clear , то есть плюс при присвоение пустого списка [ ]. При использовании clear очищается память компьютера, в которой хранились значения и везде, где на эти значения ссылалась их не будет. При присвоение пустого списка это позволяет избежать данной ситуации (очистить сам список и оставить значения там где на них ссылались).



Вывод с помощью распаковки

```
1 a,b,c = [1,2,3]
2 print(a)
3 print(b)
4 print(c)
```

Ln: 4, Col: 8

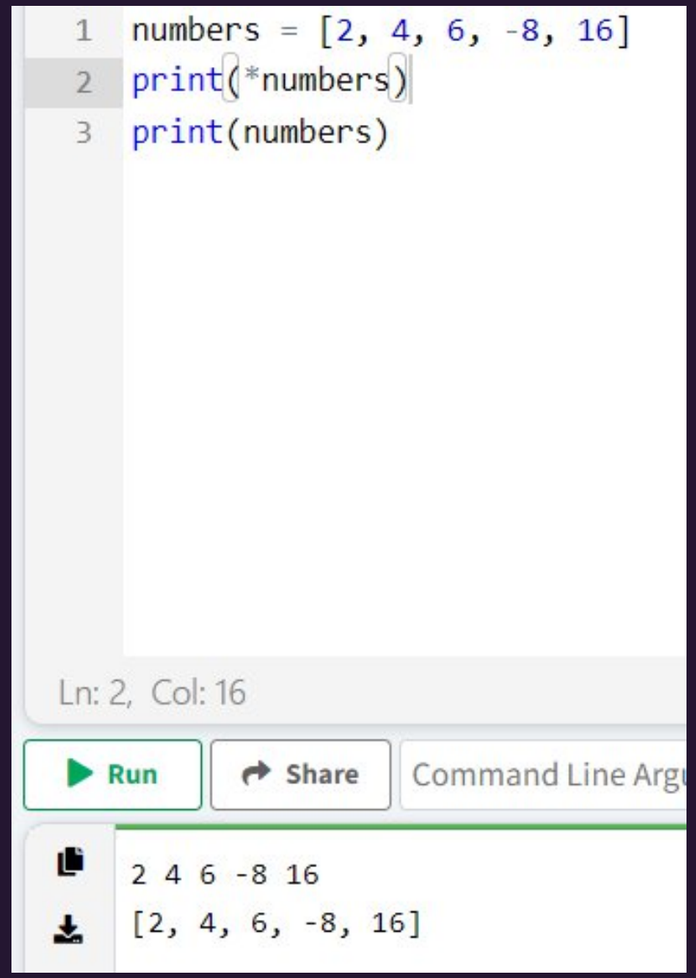
Run Share Comm

```
1
2
3
```

Распаковка списка - еще один способ быстро вывести все его элементы. Этот метод особенно удобен, когда вам нужно вывести список на одной строке.

```
numbers = [2, 4, 6, -8, 16]
print(*numbers)
```

```
numbers = [2, 4, 6, -8, 16]
print(numbers)
```



Разный вывод. Для распаковки перед именем списка указывается знак «*».

Элементы списка на экране отображаются через один символ пробела.

Оператор del

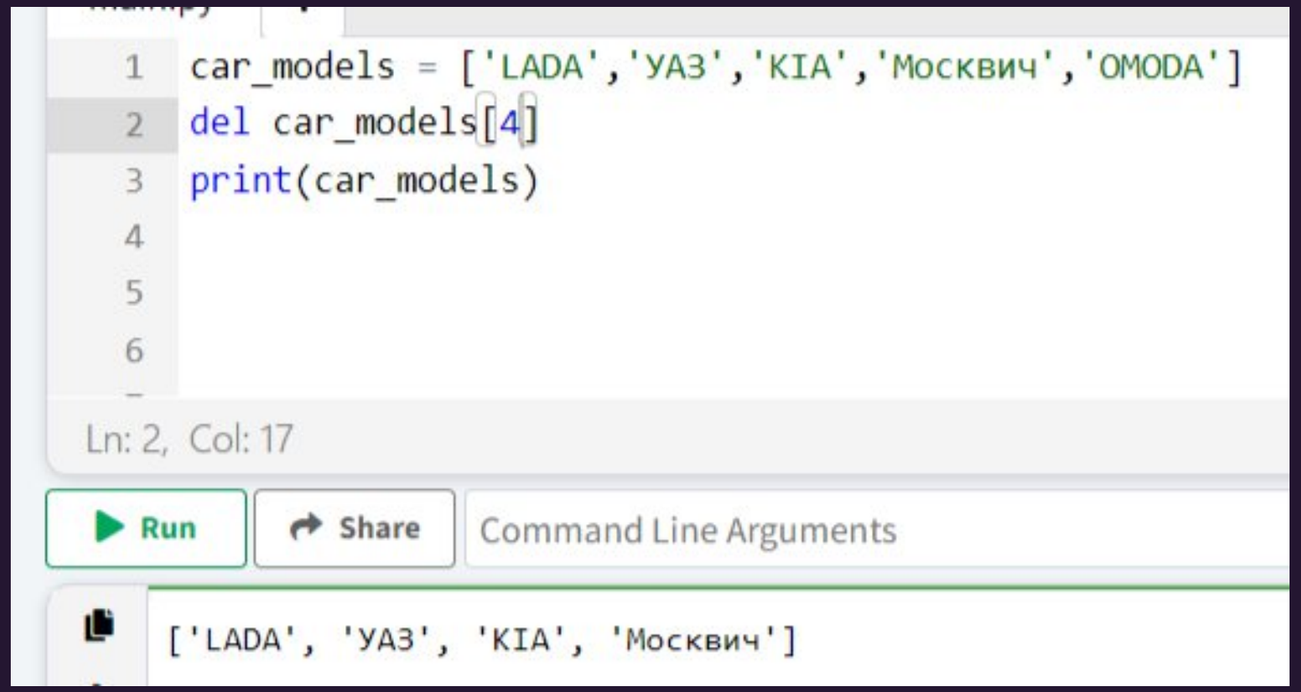
```
                  0       1       2       3            4
car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']

del car_models[4]

print(car_models)

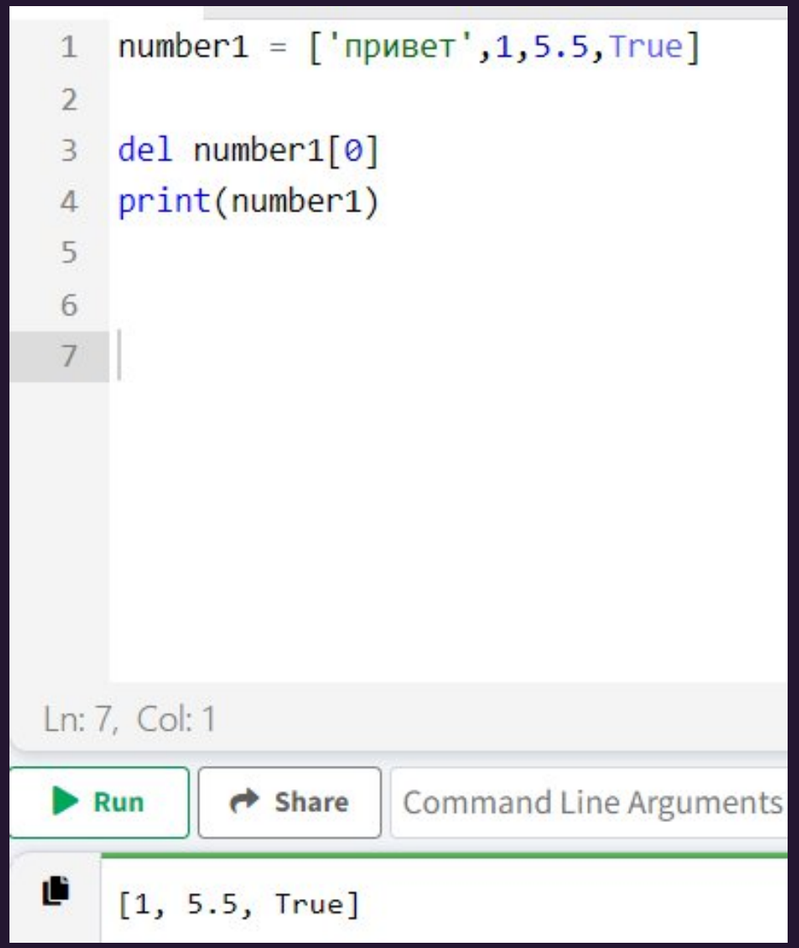

#Вывод

['LADA', 'УАЗ', 'KIA', 'Москвич']
```



Оператор del - позволяет удалять элементы из списка по **определенному индексу**. Для этого нужно указать требуемый элемент списка после оператора del.

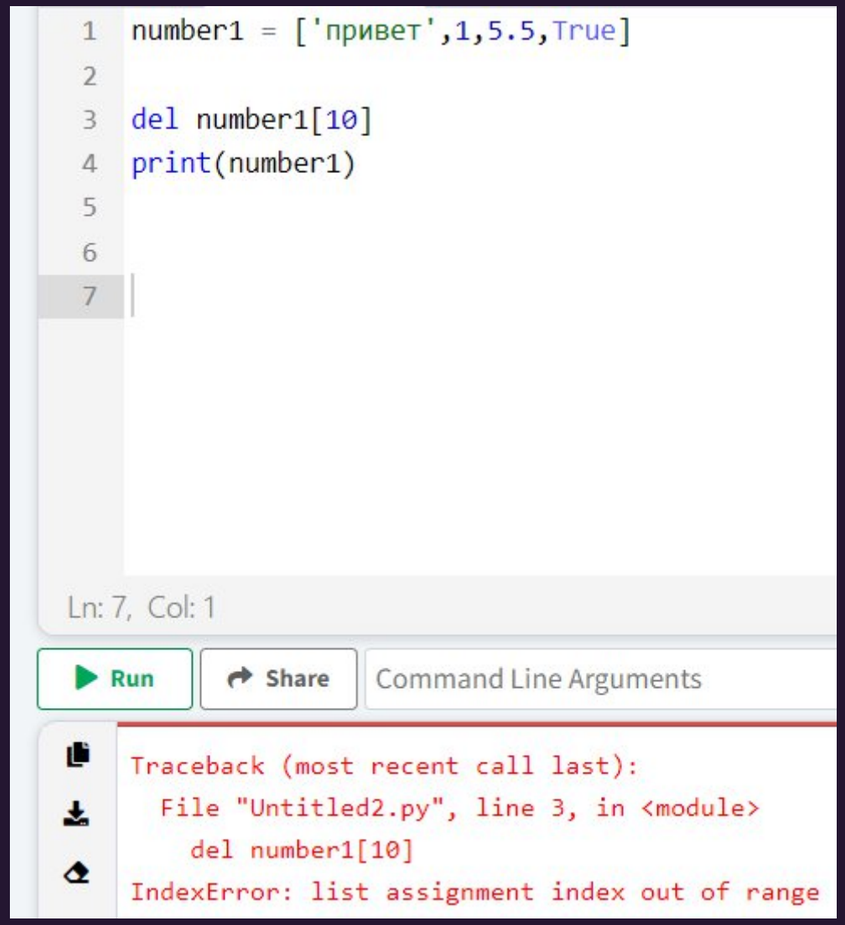
Оператор del



Оператор del - позволяет удалять элементы из списка по **определенному индексу**.

Для этого нужно указать требуемый элемент списка после оператора del.

Оператор del



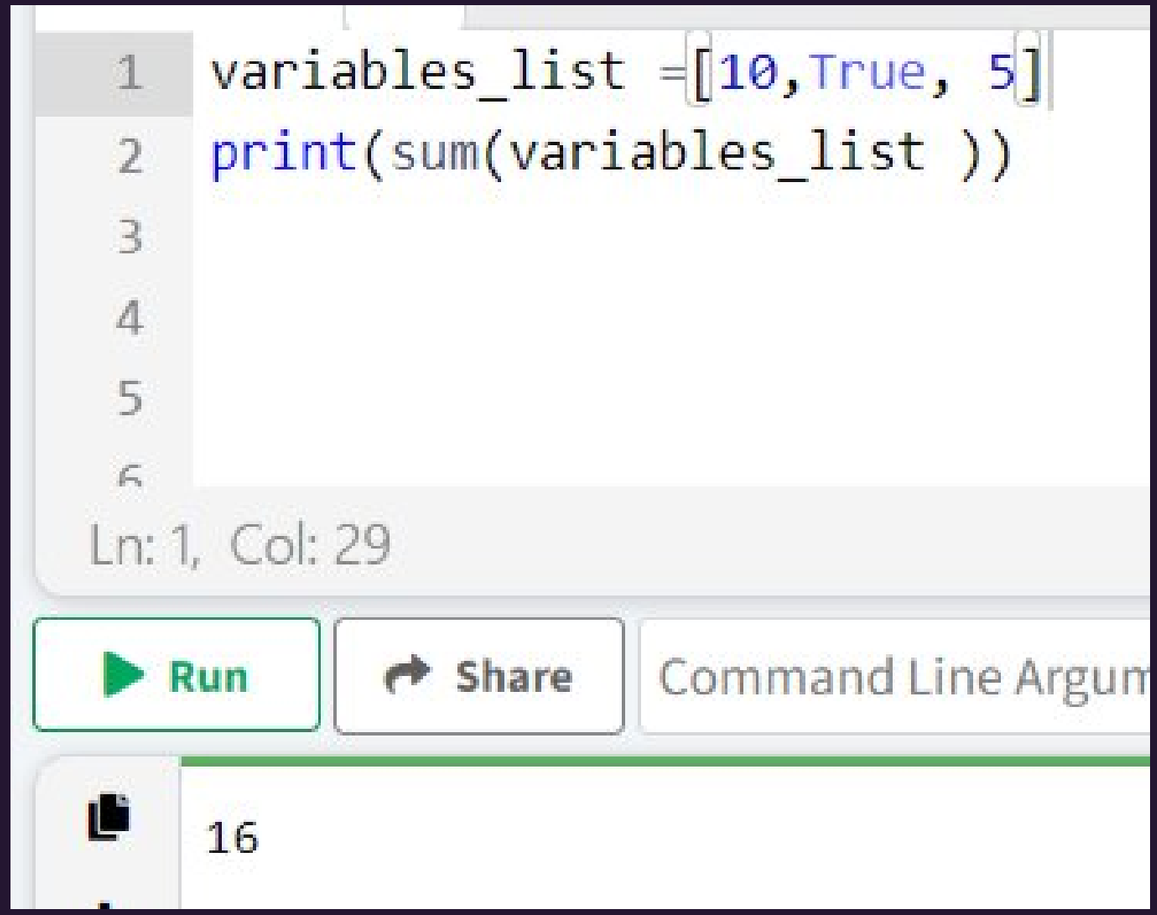
Оператор del - позволяет удалять элементы из списка по **определенному индексу**!

Заданный индекс [10] отсутствует в списке number, поэтому ошибка!!!

Функция sum()

```
variables_list =[10,True, 5]
print(sum(variables_list ))

#Вывод
16
```



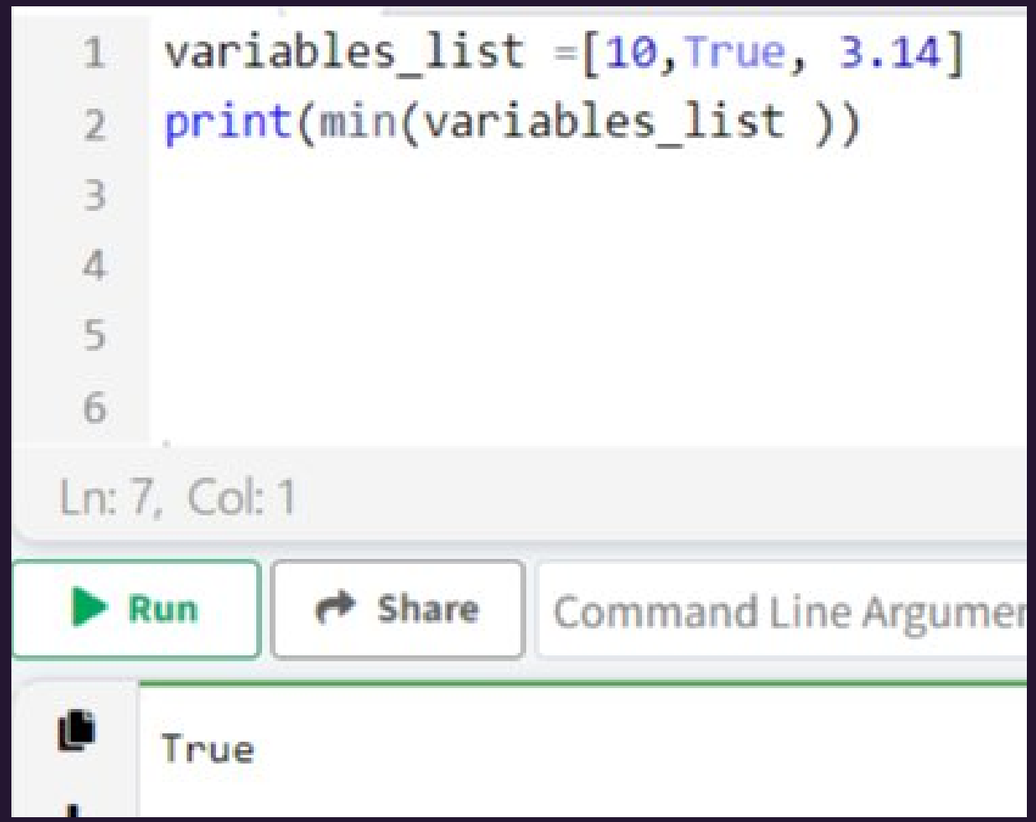
True = 1

Функция sum() возвращает сумму элементов числового списка. Она полезна для быстрого подсчета суммы значений в списке.

Функция min()

```
variables_list =[10,True, 3.14]
print(min(variables_list ))

#Вывод
True
```



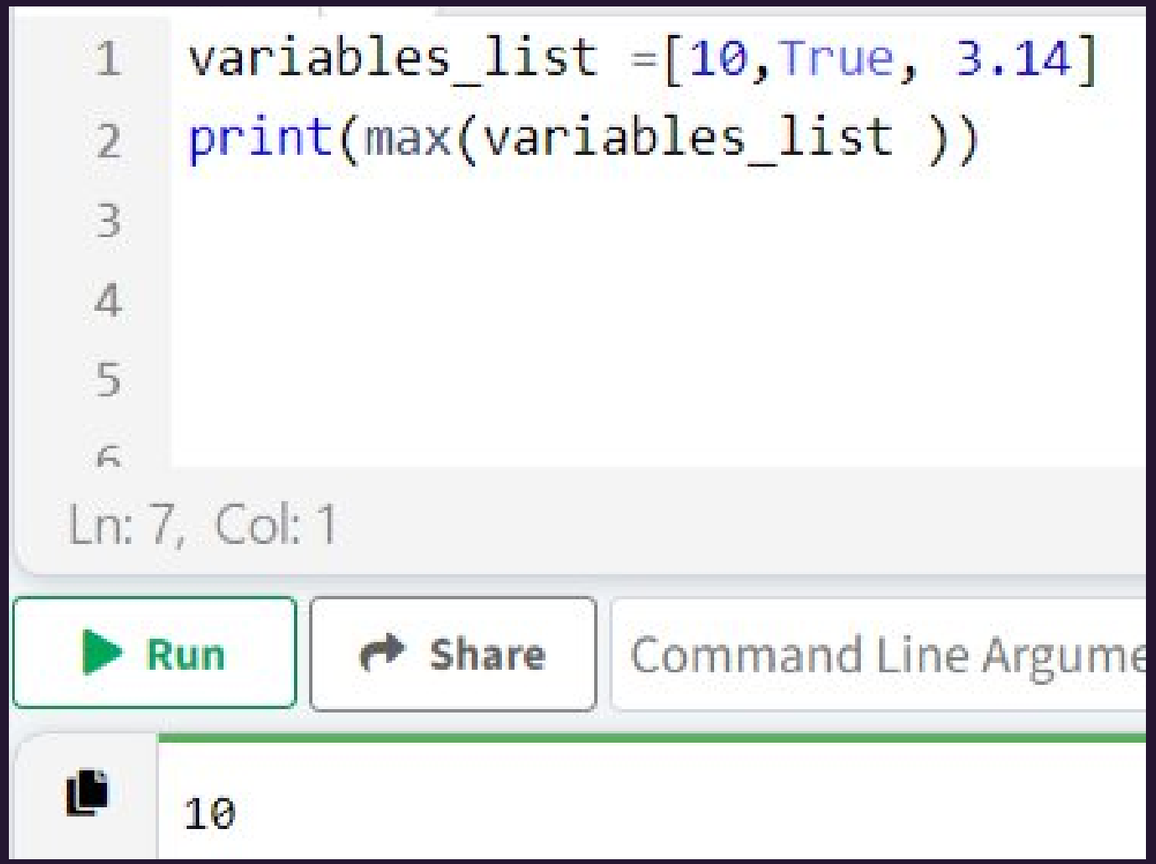
True = 1

Функция min() возвращает минимальный элемент в списке. Она помогает найти наименьшее значение в наборе данных.

Функция max()

```
variables_list =[10,True, 3.14]
print(max(variables_list ))

#Вывод
10
```

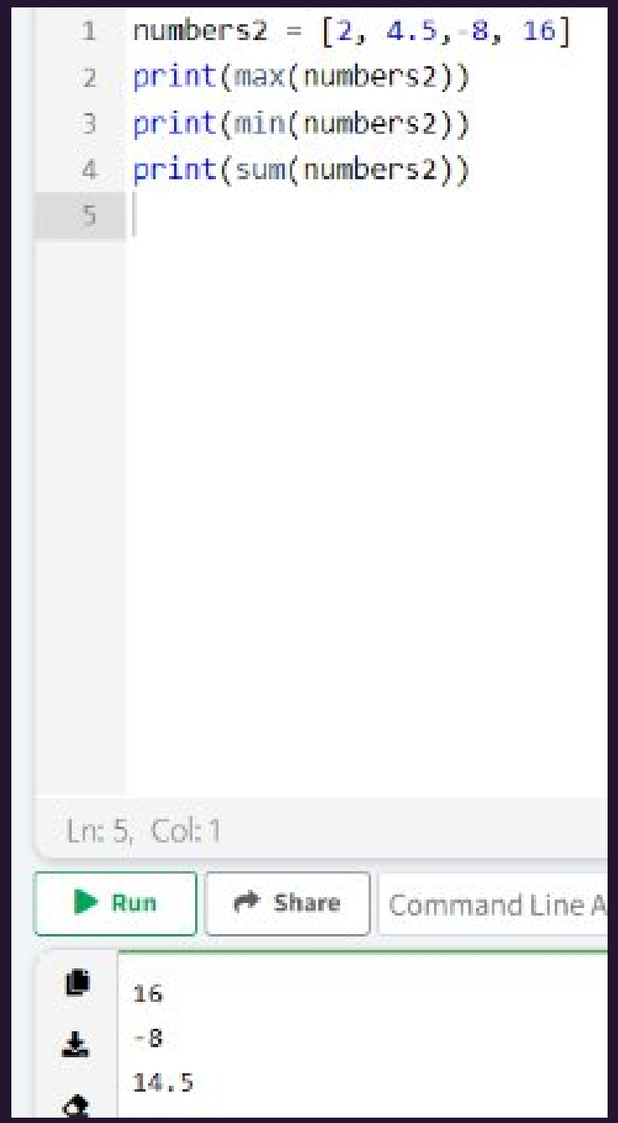


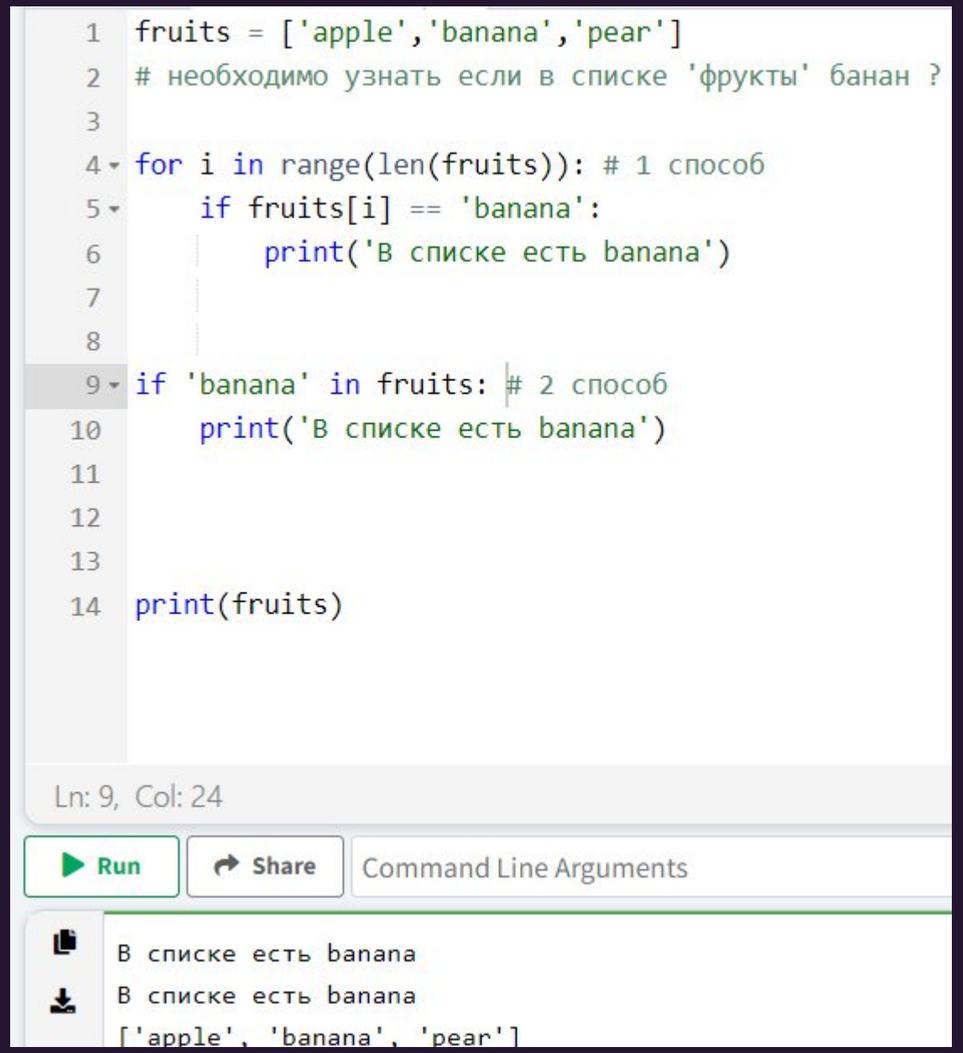
True = 1

Функция max() возвращает максимальный элемент в списке. Она полезна для определения наибольшего значения в наборе данных.

```
numbers2 = [2, 4.5,-8, 16]
print(max(numbers2))
print(min(numbers2))
print(sum(numbers2))

#Вывод
16
 -8
14.5
```





Оператор in позволяет проверить, содержится ли определенный элемент в списке. Он возвращает True, если элемент найден, и False в противном случае. В списках это проверка наличия элемента. В словарях проверка наличия ключа. В строках проверка наличия подстроки. В множествах проверка наличия элемента множества.

Дополнительные методы списков

1

sort()

Метод sort() сортирует элементы списка в порядке возрастания. Он может принимать необязательные аргументы, такие как ключевая функция и направление сортировки.

2

count()

Метод count() возвращает количество вхождений указанного элемента в списке. Это полезно для анализа частоты появления элементов.

3

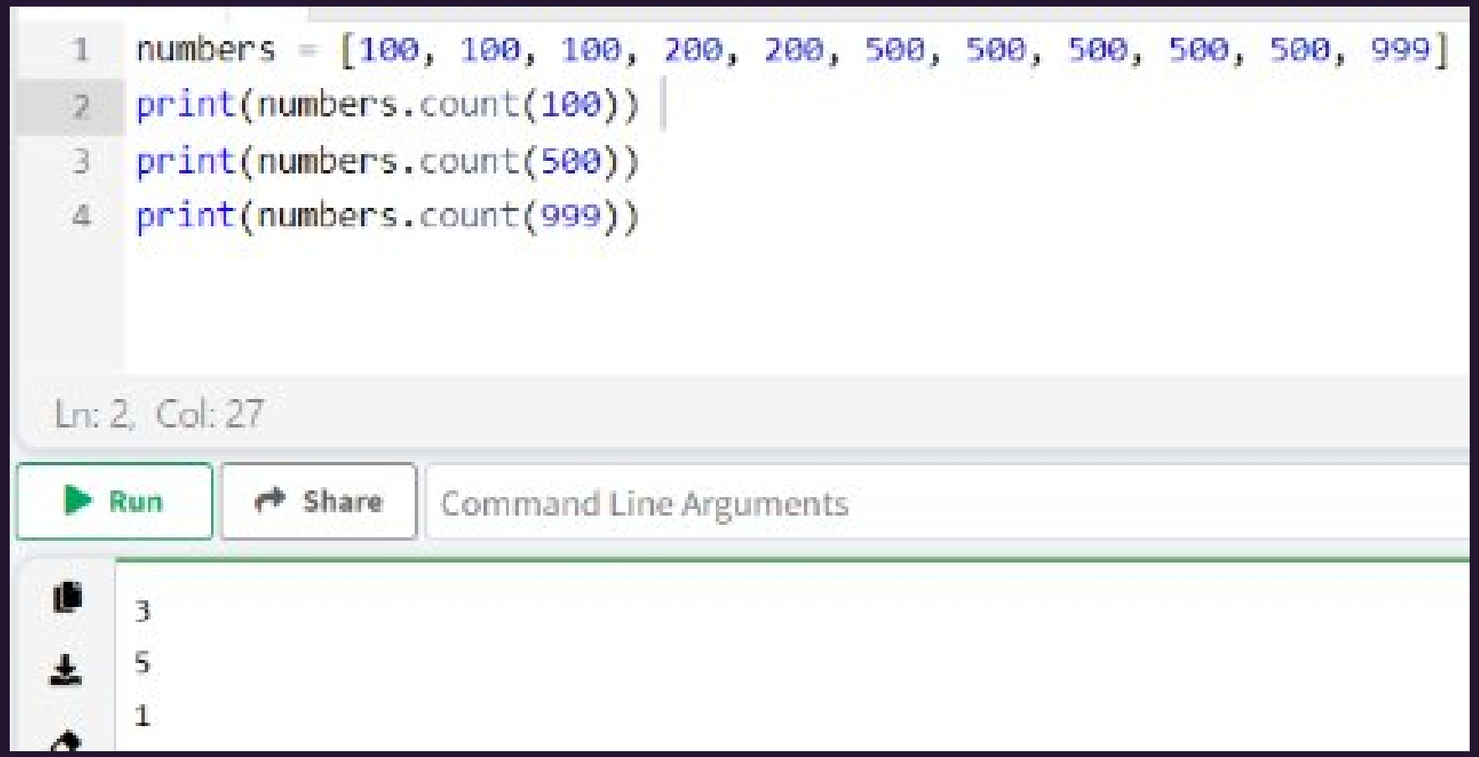
Другие методы

Python предоставляет множество других методов для работы со списками, таких как index(), reverse(), copy() и extend(). Изучение этих методов позволит вам более эффективно манипулировать и обрабатывать списки.

Метод count()

```
numbers = [100, 100, 100, 200, 200, 500, 500, 500, 500, 500, 999]
print(numbers.count(100))
print(numbers.count(200))
print(numbers.count(500))
print(numbers.count(999))

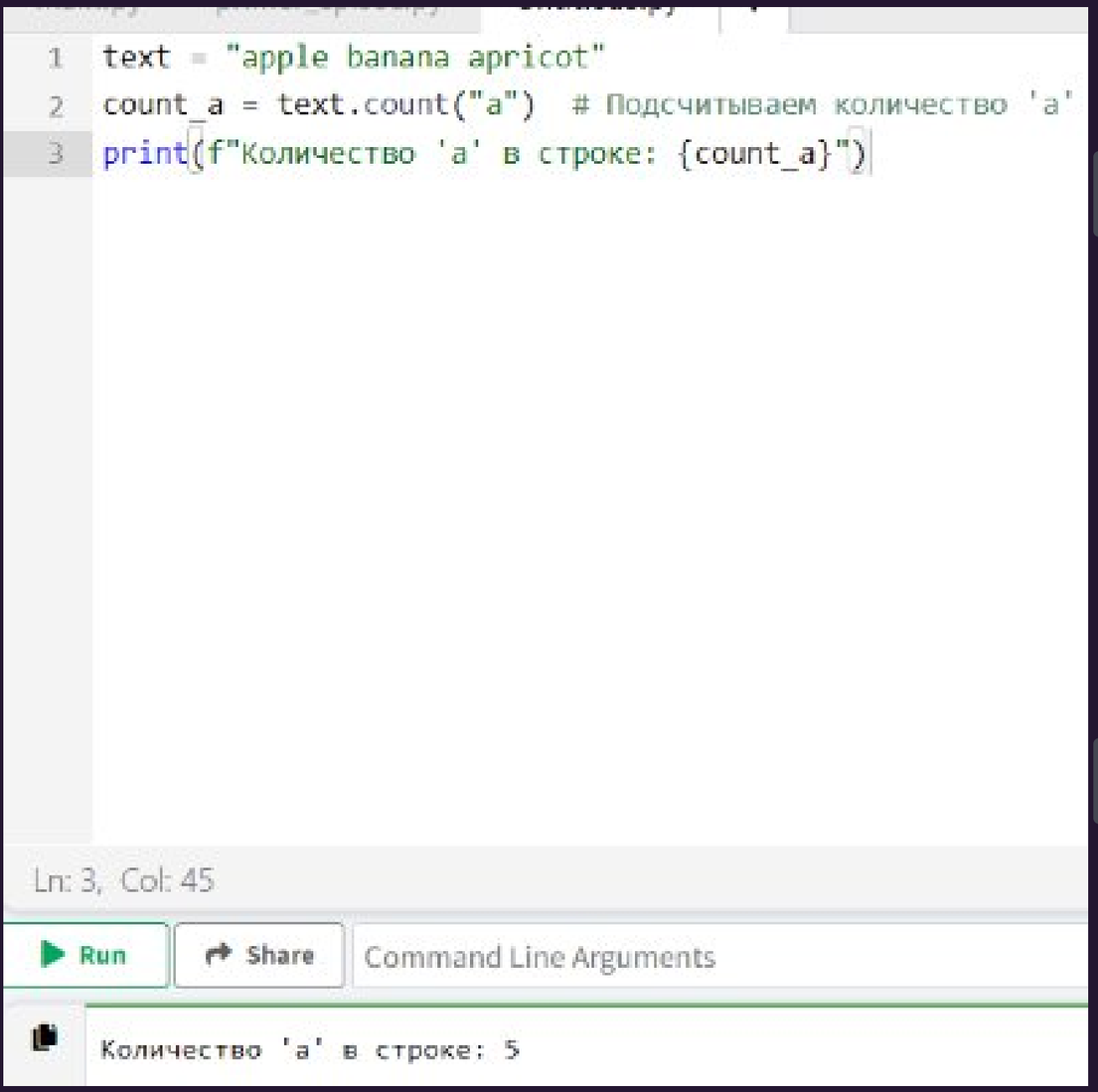

#Вывод
3
2
5
1
```



Метод **count()** - посчитаем элементы списка

Результат **count()** - это целое число, показывающее, сколько раз найдена искомая подстрока.

Метод count()



1 Подсчет символов

Метод **count()** возвращает количество вхождений заданной подстроки в строке. Это полезно для проверки частоты появления определенных символов или слов.

2 Регистрозависимость

По умолчанию **count()** учитывает регистр. Можно передать дополнительный аргумент, чтобы сделать поиск регистронезависимым.

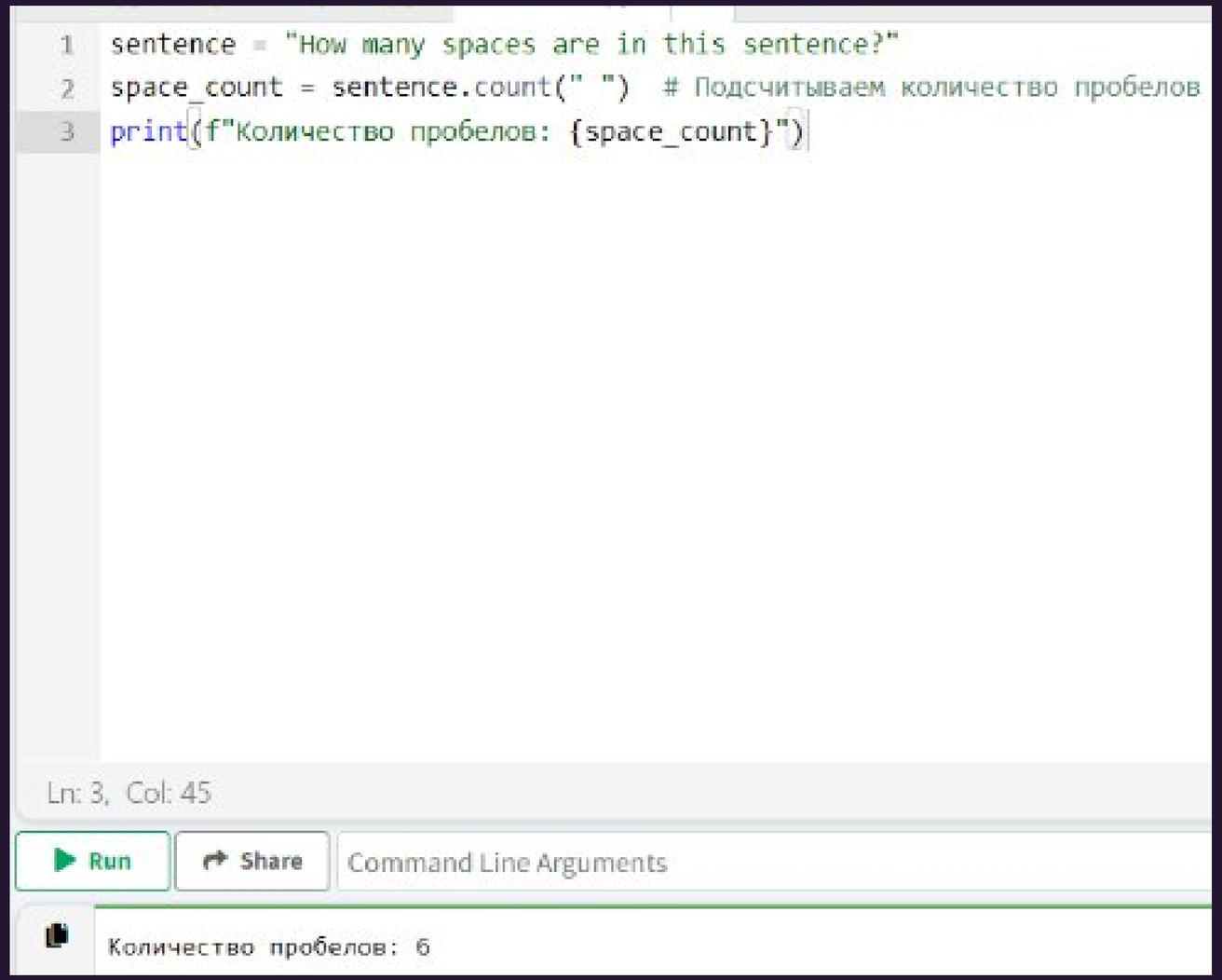
3 Счетчик вхождений

Результат **count()** - это целое число, показывающее, сколько раз найдена искомая подстрока. Это позволяет анализировать частоту встречаемости элементов в тексте.

Метод count()

```
sentence = "How many spaces are in this sentence?"
space_count = sentence.count(" ")
print(f"Количество пробелов: {space_count}")


# Вывод
Количество пробелов: 6
```



Метод **count()** возвращает количество вхождений заданной подстроки в строке. Это полезно для проверки частоты появления определенных символов или слов.

Метод **sort()**

```
numbers = [10, 2, 11, 9, 3, 24, 57, 78]

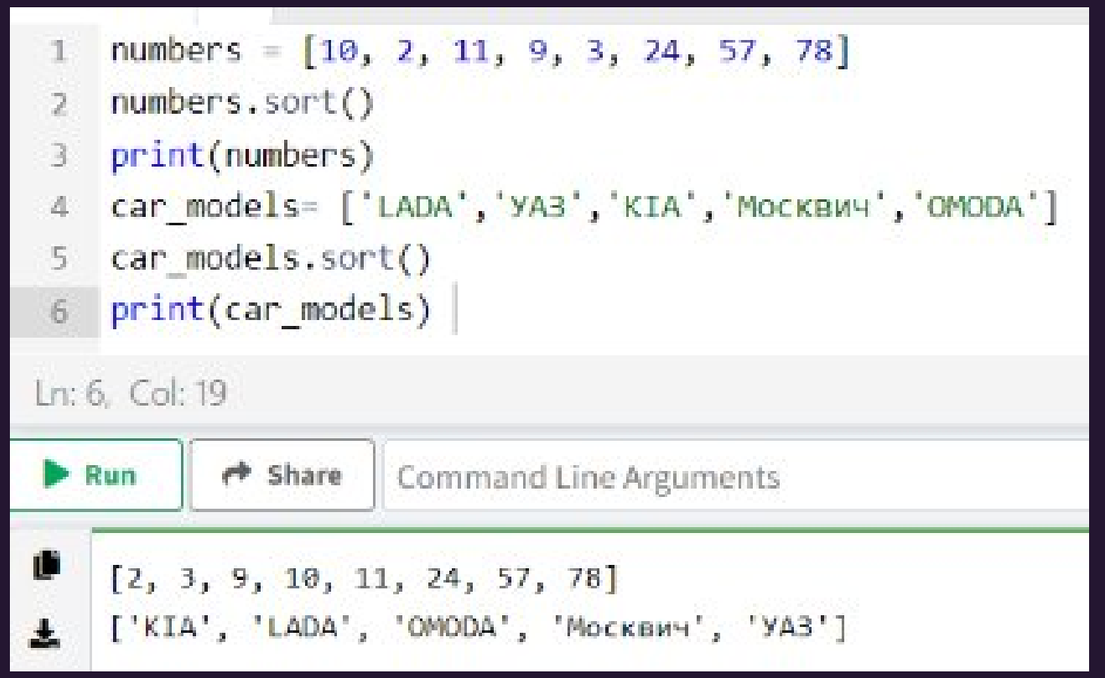
numbers.sort()

print(numbers)

car_models= ['LADA','УАЗ','KIA','Москвич','OMODA']

car_models.sort()

print(car_models)
```



#Вывод

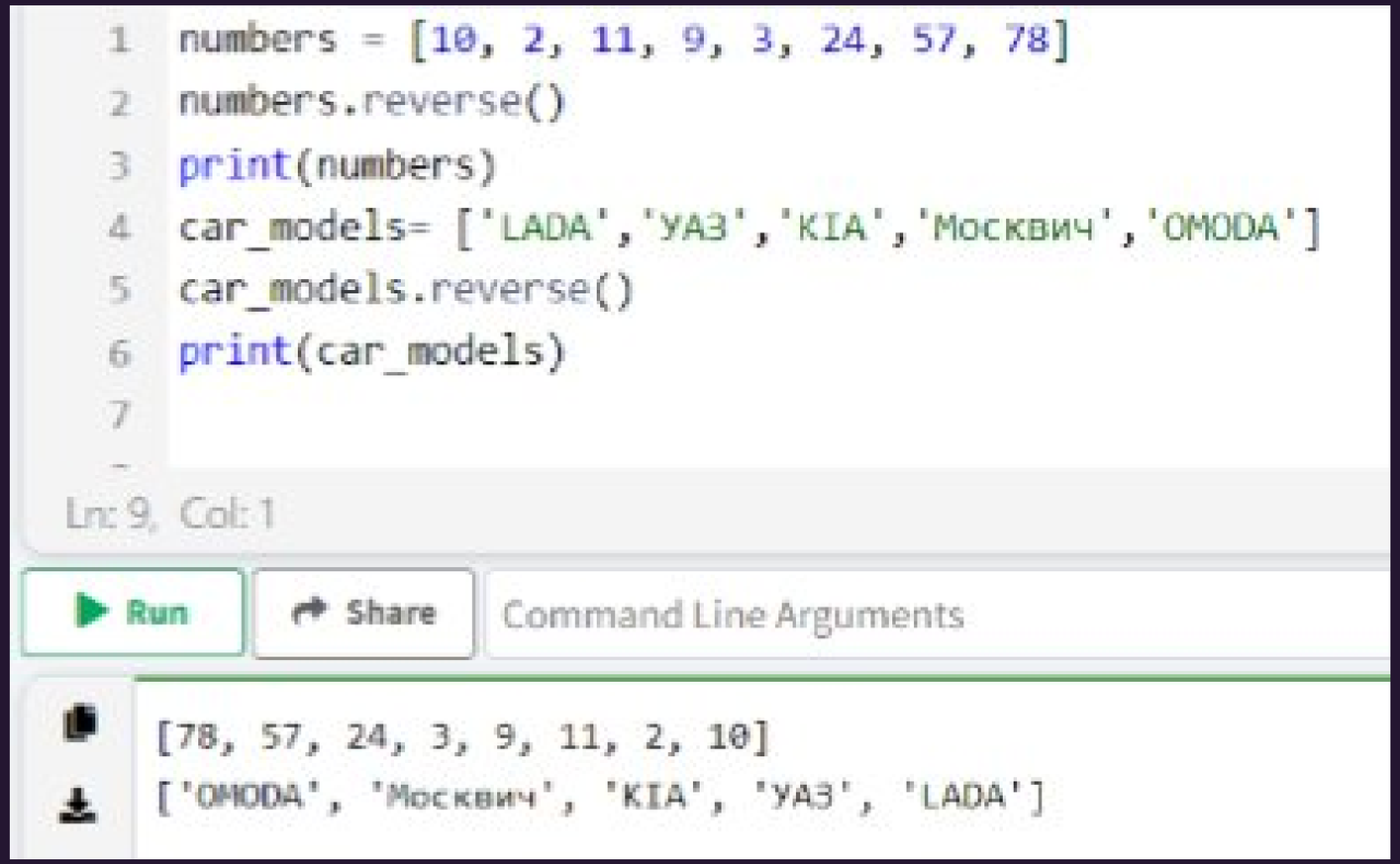
[2, 3, 9, 10, 11, 24, 57, 78]

['KIA', 'LADA', 'OMODA', 'Москвич', 'УАЗ']

Метод **sort()** **-** сортирует список по возрастанию значений его элементов.

Метод **reverse()**

```
numbers = [10, 2, 11, 9, 3, 24, 57, 78]
numbers.reverse()
print(numbers)
car_models= ['LADA','УАЗ','KIA','Москвич','OMODA']
car_models.reverse()
print(car_models)
```



#Вывод

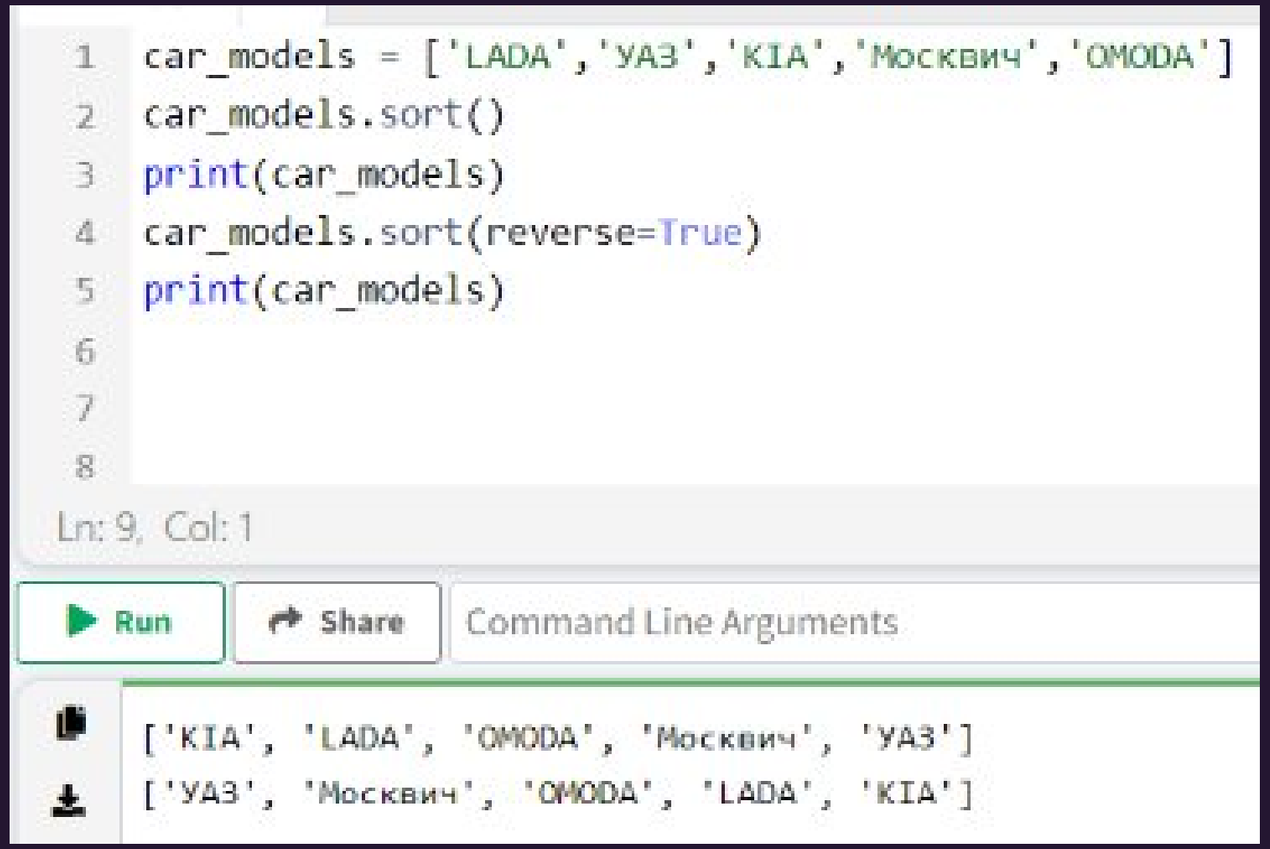
[78, 57, 24, 3, 9, 11, 2, 10]

['OMODA', 'Москвич', 'KIA', 'УАЗ', 'LADA']

Метод **reverse() -** изменить порядок элементов на обратный (перевернуть список).

Метод sort() и reverse()

```
car_models = ['LADA','УАЗ','KIA','Москвич','OMODA']
car_models.sort()
print(car_models)
car_models.sort(reverse=True)
print(car_models)
```



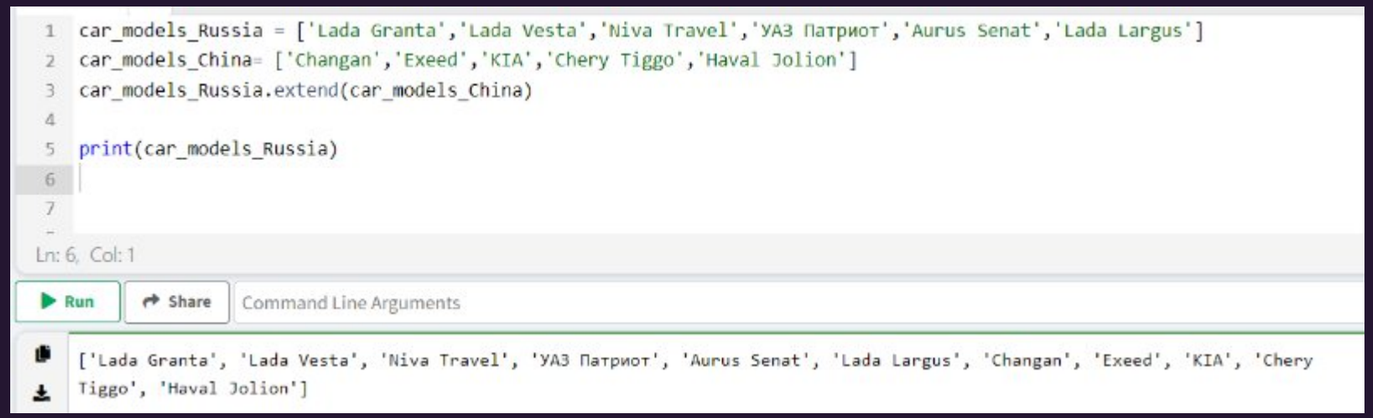

```
#Вывод
['KIA', 'LADA','OMODA', 'Москвич', 'УАЗ']
['УАЗ', 'Москвич', 'OMODA', 'LADA', 'KIA']
```

Метод **reverse()** - изменить порядок элементов на обратный (перевернуть список).

Метод extend()

```
car_models_Russia = ['Lada Granta','Lada Vesta','Niva Travel','УАЗ Патриот','Aurus Senat','Lada Largus']
car_models_China= ['Changan','Exeed','KIA','Chery Tiggo','Haval Jolion']
car_models_Russia.extend(car_models_China)


print(car_models_Russia)
```



#Вывод

['Lada Granta', 'Lada Vesta', 'Niva Travel', 'УАЗ Патриот', 'Aurus Senat', 'Lada Largus', 'Changan', 'Exeed', 'KIA', 'Chery Tiggo', 'Haval Jolion']

Метод extend() для *объединения* двух списков. Этот метод вызывается для одного списка, а в качестве аргумента ему передается другой список, extend() *записывает* в *конец первого* из них начало второго.

Метод index()

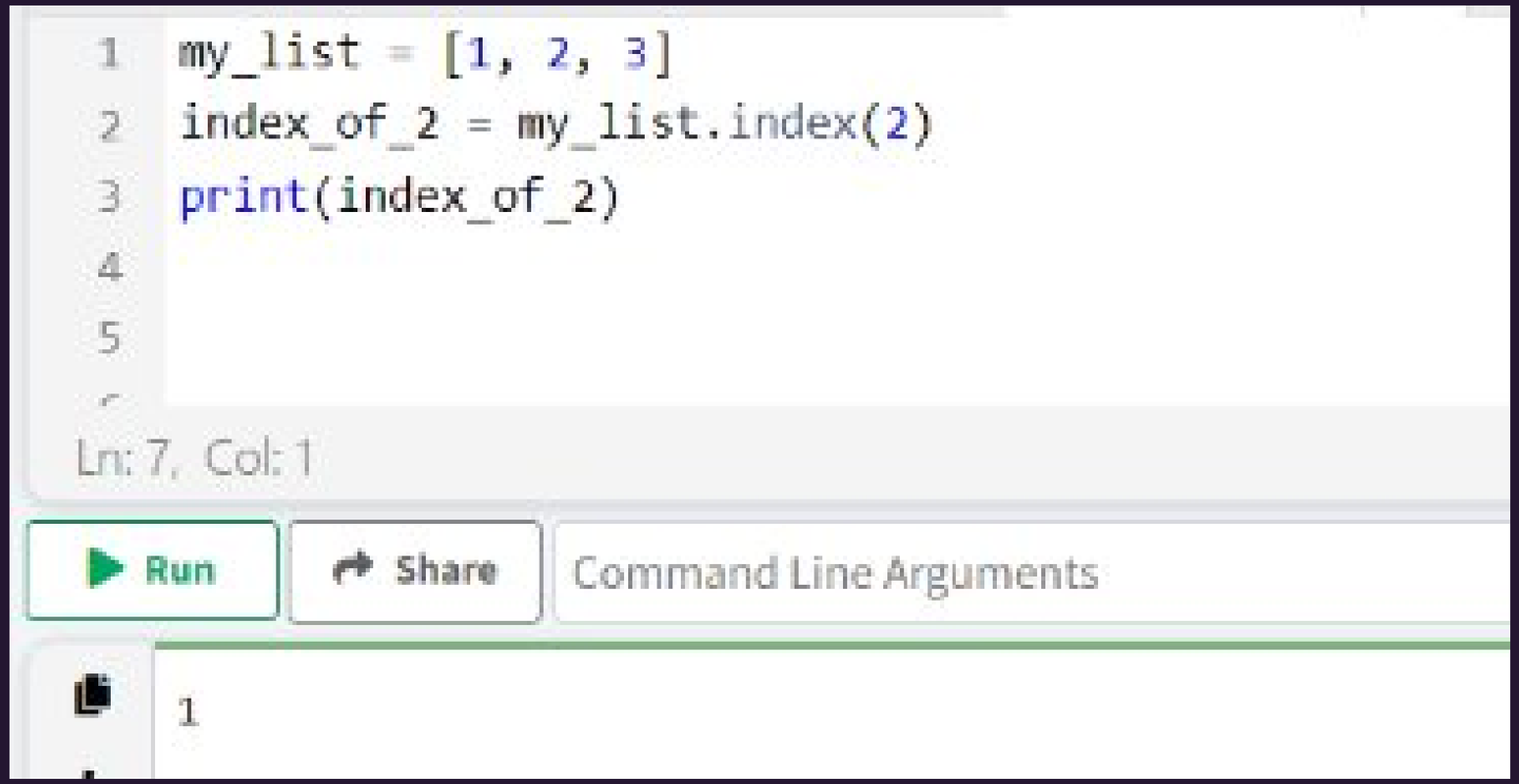
```
          0  1  2
my_list = [1, 2, 3]

index_of_2 = my_list.index(2)

print(index_of_2)


#Вывод

1
```



Метод index() - возвращает индекс элемента. Передаем в **index()** значение элемента, а метод возвращает его индекс.

Метод index()

```
                0              1              2              3              4              5
car_models = ['Lada Granta','Lada Vesta','Niva Travel','УАЗ Патриот','Aurus Senat','Lada Largus']

print(car_models.index('Lada Vesta'))

print(car_models.index('Niva Travel'))

print(car_models.index('Aurus Senat'))
```




```
#Вывод

>>> 1

>>> 2

>>> 4
```

Метод index() - возвращает индекс элемента. Передаем в **index()** значение элемента, а метод возвращает его индекс.

Списки vs. Строки

Различия

Списки и строки в Python имеют ряд отличий. Строки - это неизменяемые последовательности символов, в то время как списки - изменяемые коллекции произвольных объектов.

Методы join() и split()

Метод join() позволяет объединить элементы списка в одну строку. Метод split() разбивает строку на список, используя указанный разделитель.

Применение

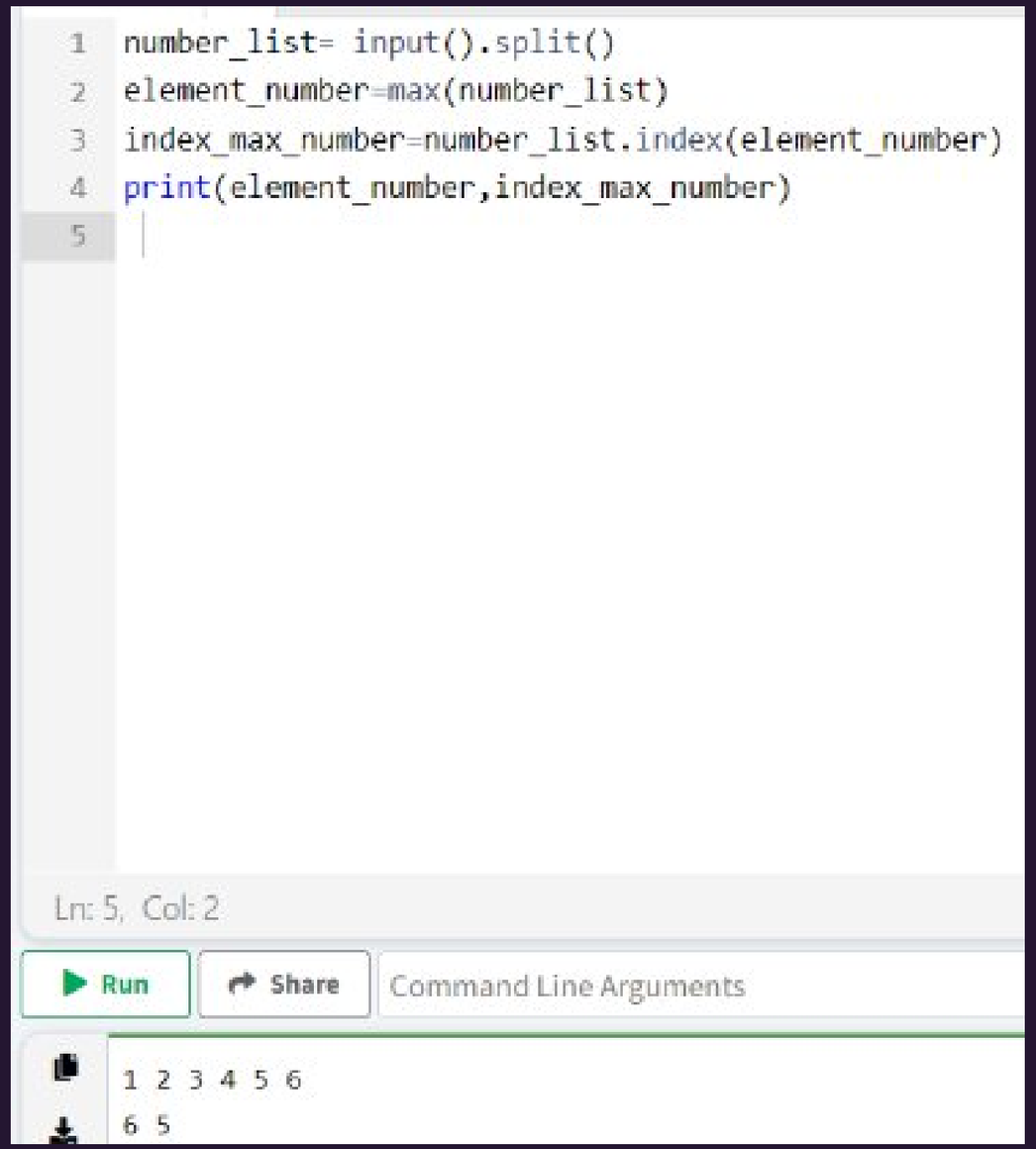
Знание различий между списками и строками, а также методов join() и split() помогает эффективно работать с текстовыми данными и преобразовывать их между различными структурами.



Метод split()

```
# Найти наибольший элемент в списке и индекс этого элемента  в списке.
number_list = input('Введите ряд чисел').split()
element_number = max(number_list)
index_max_number = number_list.index(element_number)
print(element_number, index_max_number)


 #Вывод
1 2 3 4 5 6
6 5
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()

```
# Количество положительных элемента  в списке.

numbers_str = input('Введите ряд чисел: ')


numbers = list(map(int, numbers_str.split()))


count_positive_numbers = sum(1 for num in numbers if num > 0)

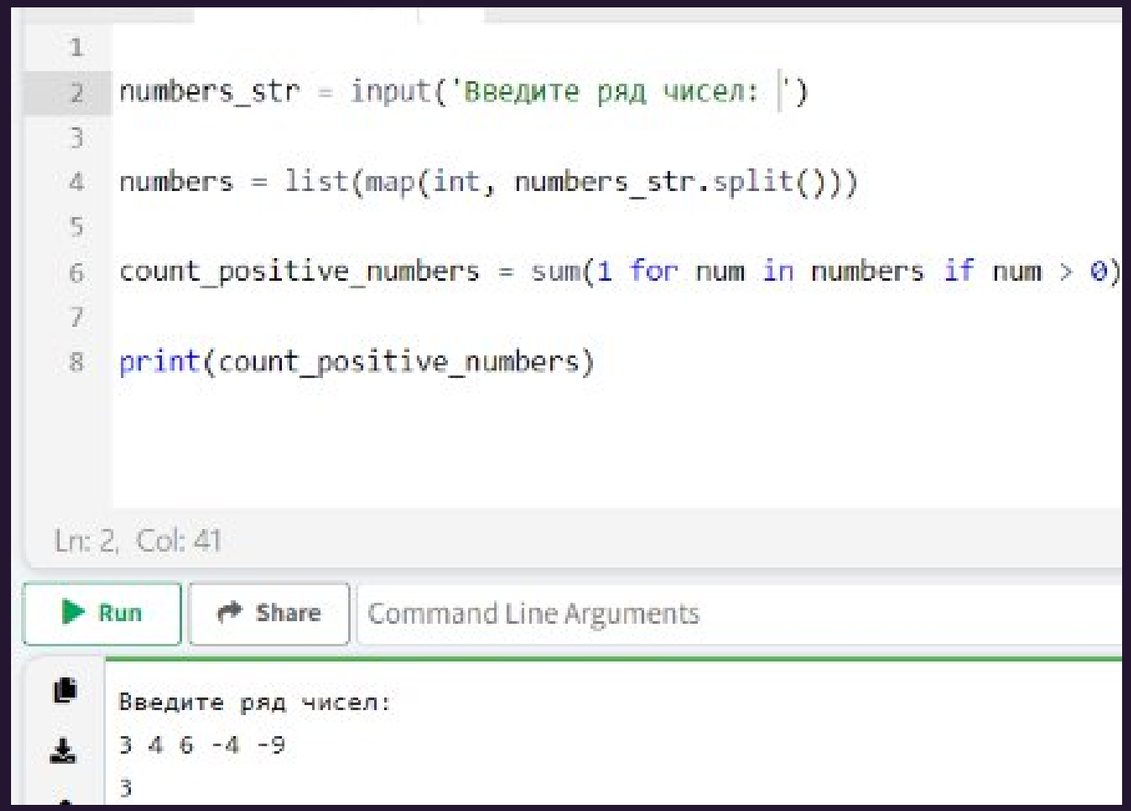

print(count_positive_numbers)


 #Вывод

Введите ряд чисел:

3 4 6 -4 -9

3
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

```
# Количество положительных элемента  в списке.
numbers_str = input('Введите ряд чисел: ')

numbers = list(map(int, numbers_str.split()))

count_positive_numbers = sum(1 for num in numbers if num > 0)

print(count_positive_numbers)
```

Цифра 1 имеет важное значение. Это не просто число, а часть так называемого **генератора списков** (list comprehension).

Генератор списков позволяет создавать новый список на основе существующего, выполняя определённые условия и операции над каждым элементом исходного списка.

В данном случае генератор списков создаёт новый список, где каждый элемент равен 1, если соответствующий элемент исходного списка больше нуля (num > 0).

Этот генератор выглядит так: [1 for num in numbers if num > 0].
Затем функция sum() применяет операцию сложения ко всем элементам этого нового списка и возвращает итоговую сумму.
То есть sum([1, 1, 1]) равно 3, а sum([1, 1]) равно 2.

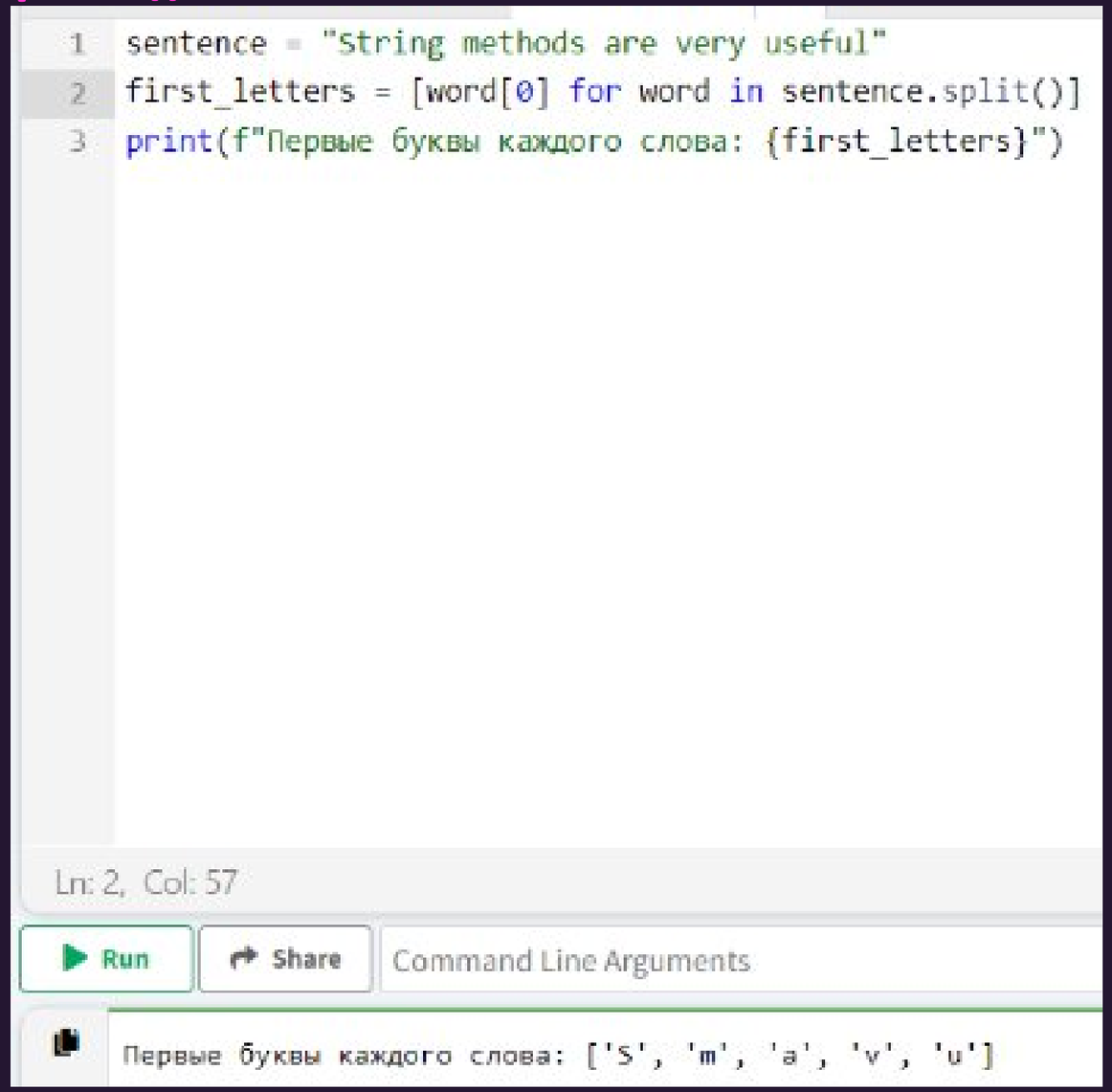
Таким образом, использование 1 внутри генератора списков необходимо для создания списка, содержащего количество положительных элементов исходного списка.

Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()

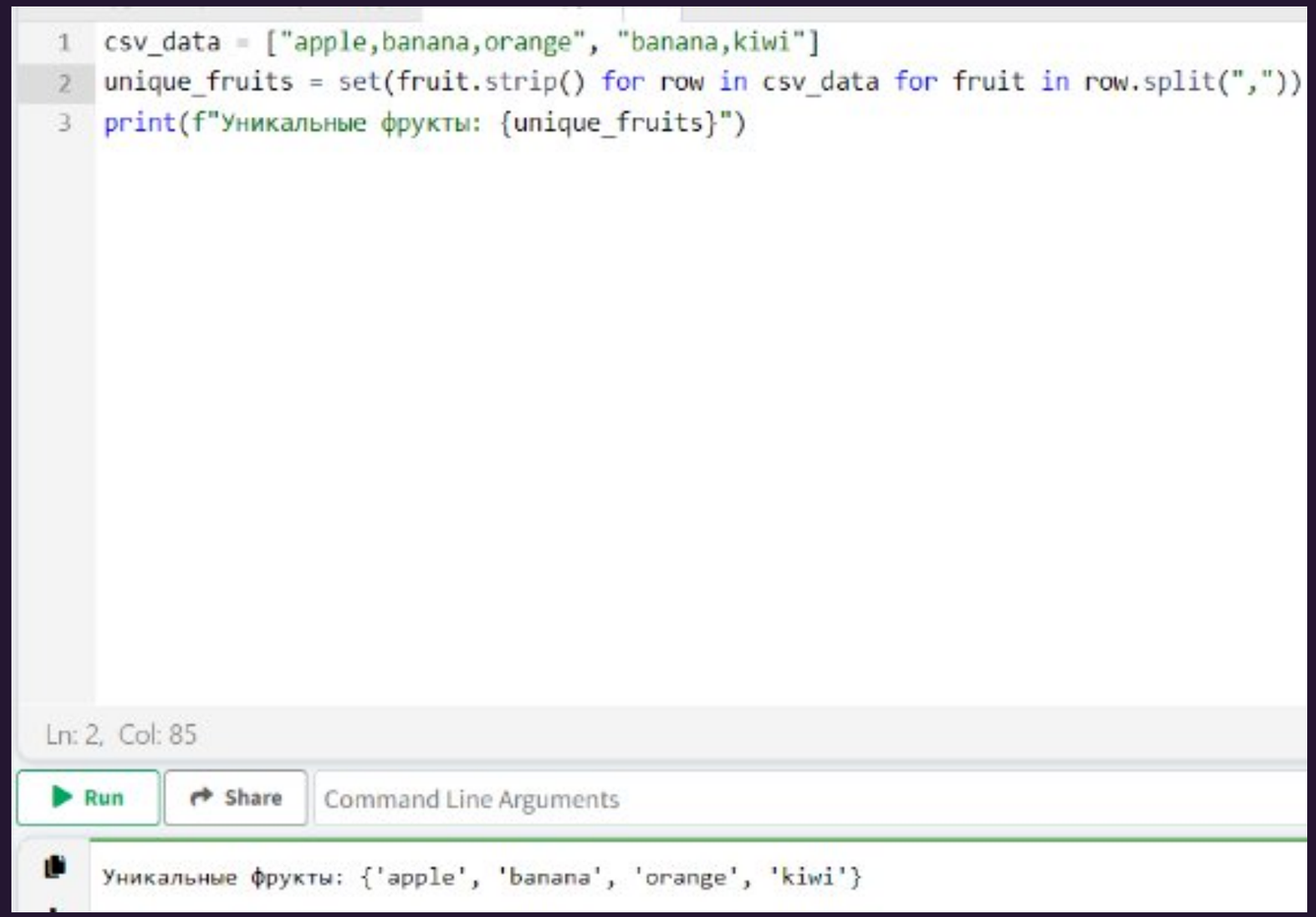
```
# Получаем первые буквы каждого слова


sentence = "String methods are very useful"
first_letters = [word[0] for word in sentence.split()]
print(f"Первые буквы каждого слова: {first_letters}")
#Вывод
Первые буквы каждого слова: ['S', 'm', 'a', 'v', 'u']
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()




```
# Разделяем по запятой

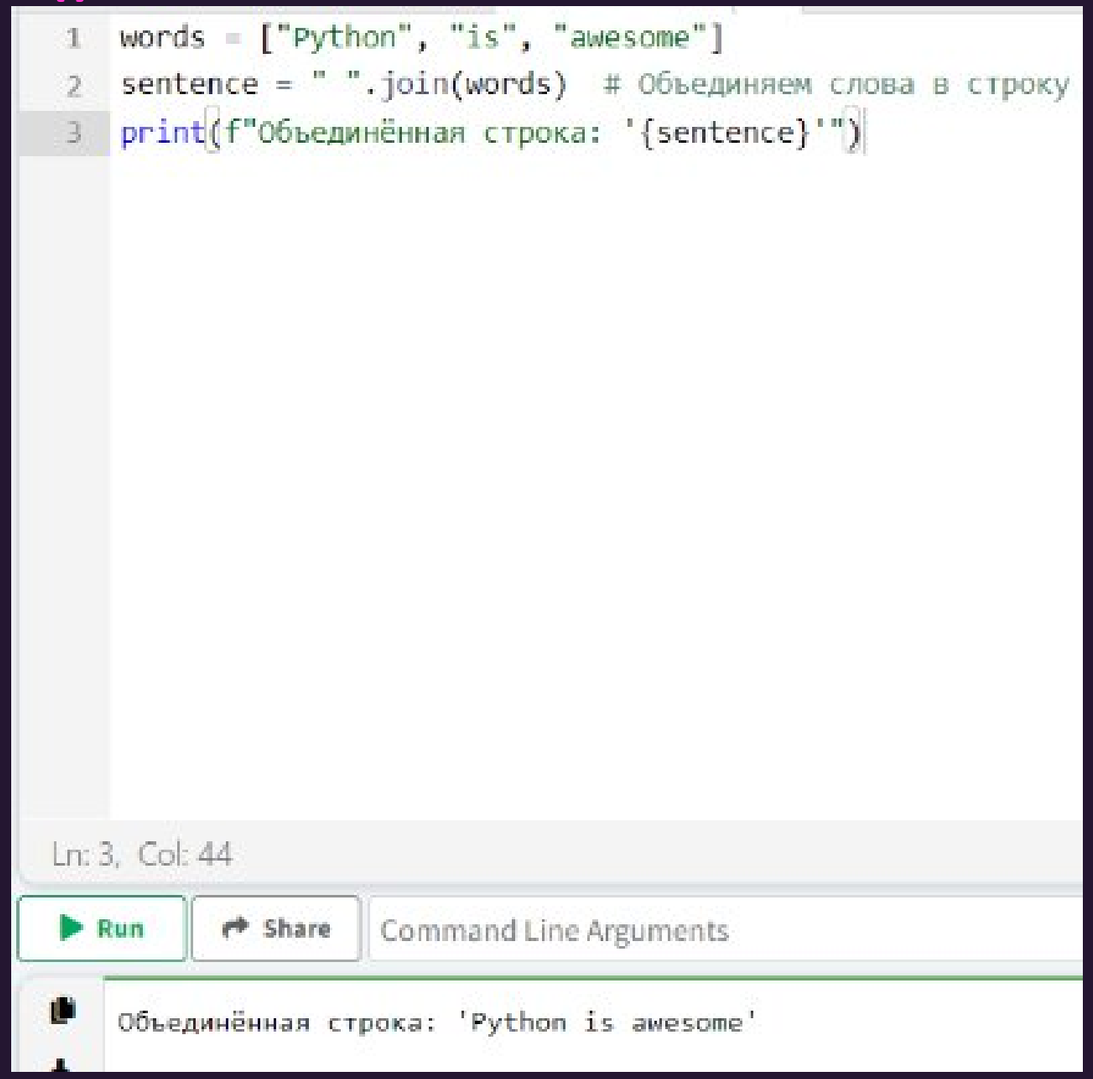

csv_data = ["apple,banana,orange", "banana,kiwi"]
unique_fruits = set(fruit.strip() for row in csv_data for fruit in row.split(","))
print(f"Уникальные фрукты: {unique_fruits}")
```

Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод join()

```
# Объедините список слов в одну строку, разделяя их пробелами.
words = ["Python", "is", "awesome"]
sentence = " ".join(words)  # Объединяем слова в строку
print(f"Объединённая строка: '{sentence}'")

#Вывод
Объединённая строка: 'Python is awesome '
```



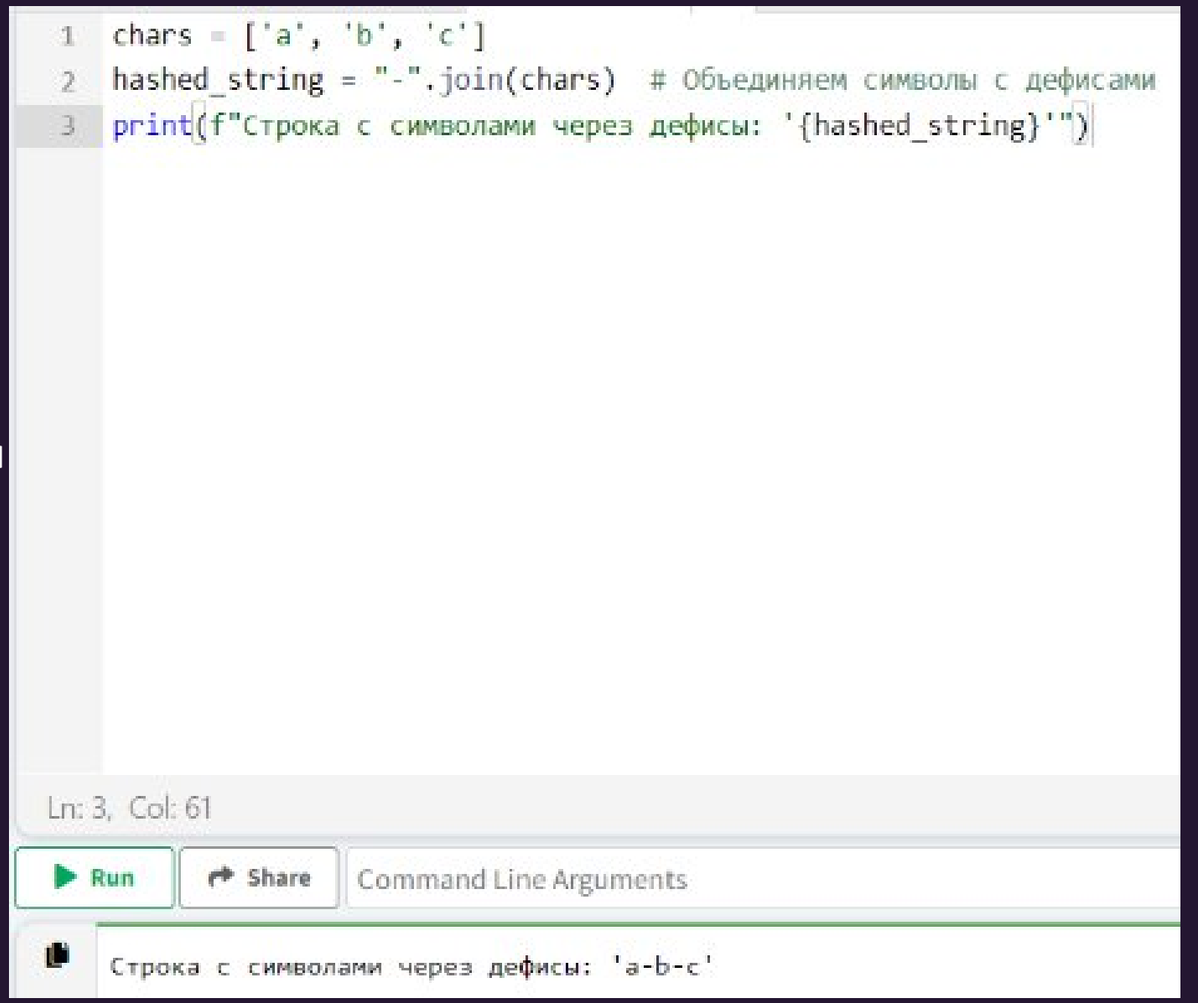
Метод **join()** используется для объединения списка строк в одну строку, используя указанный разделитель.

Это удобно для создания строк из набора отдельных компонентов.

Метод join()

```
chars = ['a', 'b', 'c']
hashed_string = "-".join(chars)  # Объединяем символы с дефисами
print(f"Строка с символами через дефисы: '{hashed_string} "')


#Вывод
Строка с символами через дефисы: 'a-b-c'
```



Метод **join()** используется для объединения списка строк в одну строку, используя указанный разделитель.